Knowledge Representation and Reasoning

Chapter 3

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Knowledge

Knowledge is a theoretical or practical understanding of a subject or a domain.

Knowledge is also the sum of what is currently known.

Types of knowledge:

Classification-based Knowledge :Ability to classify information

Decision-oriented Knowledge: Choosing the best option

Descriptive knowledge: State of some world (heuristic)

Procedural knowledge: How to do something

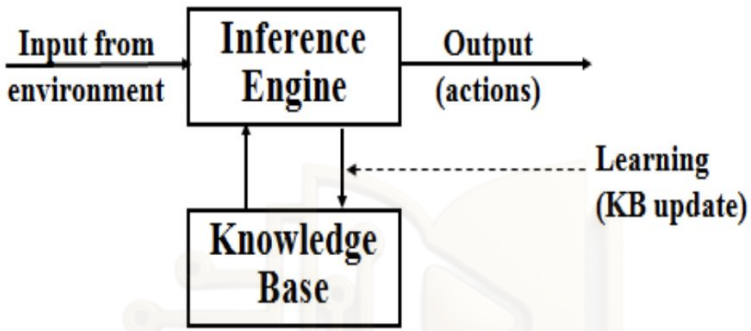
Reasoning knowledge: What conclusion is valid in what situation?

Assimilative knowledge: What its impact is?

notes.arpanadhikari7.com.np

A Knowledge-Based Agent

- A knowledge-based agent consists of a knowledge base (KB) and an inference engine (IE).



- A knowledge-base is a set of sentences of what one knows about the world.
- The Inference engine derives new sentences from the input and KB.
- The agent operates as follows:
  1. It receives percepts from environment
  2. It computes what action it should perform (by IE and KB)
  3. It performs the chosen action.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Properties for Knowledge Representation Systems: –

The following properties should be possessed by a knowledge representation system.

- Representational Adequacy
  - the ability to represent the required knowledge;
- Inferential Adequacy
  - the ability to manipulate the knowledge represented to produce new knowledge corresponding to that inferred from the original;
- Inferential Efficiency
  - the ability to direct the inferential mechanisms into the most productive directions by storing appropriate guides;
- Acquisitional Efficiency
  - the ability to acquire new knowledge using automatic methods wherever possible rather than reliance on human intervention.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

The objective of knowledge representation is to express the knowledge about the world in a computer-tractable form.

This computer- tractable form of knowledge helps the agent to identify patterns of good reasoning and patterns of bad reasoning, so the agent know which to follow and which to avoid.

A formal language is required to represent knowledge in a computer tractable form and reasoning processes are required to manipulate this knowledge to deduce new facts.

Key aspects of knowledge representation languages are: –

- Syntax: describes how sentences are formed in the language.
- Semantics: describes the meaning of sentences, what is it the sentence refers to in the real world.
- Proof Theory(Inference method): Set of rules for generating new sentences that are necessarily true given that the old sentences are true

Available on notes.arpanadhikari7.com.np

Knowledge Representation using Logic: –

Logic is defined as a formal language for expressing knowledge and ways of reasoning.

Therefore, it should have syntax, semantics and inference method.

Syntax: describes how sentences are formed in the LOGIC.

Semantics: describes the meaning of sentences.

Inference method: set of rules for generating new sentences.

notes.arpanadhikari7.com.np

Compared to natural languages (expressive but context sensitive) and programming languages (good for concrete data structures but not expressive) logic combines the advantages of natural languages and formal languages.

So, Logic is:

Concise, unambiguous, context insensitive, expressive, effective for inferences

Examples of Logics are:

Propositional logic

Predicate Logic and

Fuzzy Logic

notes.arpanadhikari7.com.np

Propositional Logic

Propositional logic is the simplest formal logic for the representation of the knowledge in terms of propositions.

Proposition is a declarative statement that is either true or false but not both.

If a proposition is true, then we say it has a truth value of "true"; if a proposition is false, its truth value is "false".

Some examples of Propositions are given below :

"Man is Mortal", it returns truth value —TRUE‖

"12 + 9 = 3 – 2", it returns truth value —FALSE‖

The following sentences are not Proposition:

"A is less than 2". It is because unless we give a specific value of A, we cannot say whether the statement is true or false.

Also the sentences "Close the door", and "Is it hot outside ?"are not propositions.

notes.arpanadhikari7.com.np

Syntax of Propositional Logic

Syntax of the propositional logic defines the:

- Which symbols can be use (English: letters, punctuation)
- Rules for constructing legal sentences in the logic.
- How we are allowed to combine symbols

Symbols:

- Logical constants: true, false
- Propositional symbols: P, Q, R, S, ... , etc
- Wrapping parentheses: (…)

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

- **Atomic formulas(Sentence)**: Propositional Symbols or logical constants.

- **Literals:** atomic sentences and their negations

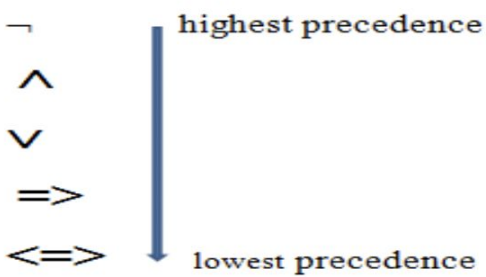
- **Complex Formulas:** can be formed by combining atomic formulas with the following connectives:

¬ ...not	[negation]
∧ ...and	[conjunction]
∨ ...or	[disjunction]
⟶ ...implies	[implication / conditional]
⟷ ..is equivalent	[biconditional]

Available on notes.arpanadhikari7.com.np

- A sentence is defined as follows:
  - A symbol is a sentence
  - If S is a sentence, then ¬S is a sentence
  - If S is a sentence, then (S) is a sentence
  - If S and T are sentences, then (S ∨ T), (S ∧ T), (S → T), and (S ↔ T) are sentences
  - A sentence results from a finite number of applications of the above rules

- **Order of precedence** of logical connectors



e.g. $\neg P \vee Q \wedge R \Rightarrow S$ is equivalent to $((\neg P) \vee (Q \wedge R)) \Rightarrow S$

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Examples of PL sentences

- P means "It is hot"
- Q means "It is humid"
- R means "It is raining"
- P ^ Q => R

  "If it is hot and humid, then it is raining"
- Q => P

  "If it is humid, then it is hot"
- Q

  "It is humid."

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Propositional Logic (PL): Semantics

- The semantics of propositional logic defines the rules for determining the truth of a sentence with respect to a particular given model (model fixes the truth value (true or false)for every proposition symbol).
  - For example, If the sentences in the knowledge base make use of the proposition symbols P, Q, and R then one possible model is:

$$\text{Model (m)} = \{ P= F, Q= F, R= T\}$$

- In propositional logic all sentences are constructed from atomic sentences and the five connectives
- So semantics of propositional logic requires rules to define the truth of atomic sentences and rules to define the truth of sentences formed with each of the five connectives

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

- How to compute the truth of atomic sentences?
  - The truth value of Atomic sentences can be determined by using the following rules :
    - True is true in every model and False is false in every model.
    - The truth value of every other proposition symbol must be specified directly in the model.
    - For example, in the model m given earlier, P is false(F).

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

- How to compute the truth of sentences formed with each of the five connectives?
  - For complex sentences, we have five rules, which hold for any sub-sentences P and Q in any model m:

$\neg P$ is true iff P is false in m.
$P \wedge Q$ is true iff both P and Q are true in m.
$P \vee Q$ is true iff either P or Q is true in m.
$P \Rightarrow Q$ is true unless P is true and Q is false in m.
$P \Leftrightarrow Q$ is true iff P and Q are both true or both false in m.

  - The above rules can be summarized as follows:

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
false	*false*	*true*	*false*	*false*	*true*	*true*
false	*true*	*true*	*false*	*true*	*true*	*false*
true	*false*	*false*	*false*	*true*	*false*	*false*
true	*true*	*false*	*true*	*true*	*true*	*true*

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Properties

- Validity(Tautology)
- Satisfiability (contingency)
- Un-Satisfiability (Contradictory)
- Equivalent
- Entailment
- Completeness
- Soundness

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

- **Validity:**
  - A sentence is valid if it is true in all models,

    e.g., *True*, $A \vee \neg A$, $A \Rightarrow A$, $(A \wedge (A \Rightarrow B)) \Rightarrow B$

  - Valid sentences are also known as tautologies. Every valid sentence is logically equivalent to True

- **Example** : Prove $[(A \rightarrow B) \wedge A] \rightarrow B$ is a tautology

  Solution: The truth table is as follows

A	B	$A \rightarrow B$	$(A \rightarrow B) \wedge A$	$[(A \rightarrow B) \wedge A] \rightarrow B$
True	True	True	True	True
True	False	False	False	True
False	True	True	False	True
False	False	True	False	True

notes.arpanadhikari7.com.np

As we can see every value of $[(A \rightarrow B) \wedge A] \rightarrow B$ is "True", it is a tautology.

Available on notes.arpanadhikari7.com.np

- **Satisfiability:**
  - A sentence is satisfiable if it is true in some model
    - E.g., A V A
  - Satisfiable sentences are also known as Contingency.

- **Example:** Prove (A∨B)∧(¬A)a contingency

Solution: The truth table is as follows

A	B	A ∨ B	¬ A	(A ∨ B) ∧ (¬ A)
True	True	True	False	False
True	False	True	False	False
False	True	True	True	True
False	False	False	True	False

As we can see every value of (A∨B)∧(¬A) has both "True" and "False", it is a contingency.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

- **Un-Satisfiability (Contradictory) :**
  - A sentence is un-satisfiable if it is true in no models
    - E.g., A ∧ ¬ A
  - Un-Satisfiable sentences are also known as contradictory sentences.
- **Example:** Prove (A∨B)∧[(¬A)∧(¬B)] is a contradiction

  Solution: The truth table is as follows :

A	B	A ∨ B	¬ A	¬ B	(¬ A) ∧ (¬ B)	(A ∨ B) ∧ [(¬ A) ∧ (¬ B)]
True	True	True	False	False	False	False
True	False	True	False	True	False	False
False	True	True	True	False	False	False
False	False	False	True	True	True	False

notes.arpanadhikari7.com.np

As we can see every value of (A∨B)∧[(¬A)∧(¬B)] is “False”, it is a contradiction.

- # Propositional Equivalences
- Two statements X and Y are logically equivalent if any of the following two conditions hold :
  - The truth tables of each statement have the same truth values.
  - The bi-conditional statement X⇔Y is a tautology.
- **Example :** Prove ¬(A∨B)and[(¬A)∧(¬B)] are equivalent

  Testing by 1st method (Matching truth table)

A	B	A ∨ B	¬ (A ∨ B)	¬ A	¬ B	[(¬ A) ∧ (¬ B)]
True	True	True	False	False	False	False
True	False	True	False	False	True	False
False	True	True	False	True	False	False
False	False	False	True	True	True	True

notes.arpanadhikari7.com.np

Here, we can see the truth values of ¬(A∨B)and[(¬A)∧(¬B)] are same, hence the statements are equivalent.

- Testing by 2nd method (Bi-conditionality)

A	B	¬ (A ∨ B)	[(¬ A) ∧ (¬ B)]	[¬ (A ∨ B)] ⇔ [(¬ A) ∧ (¬ B)]
True	True	False	False	True
True	False	False	False	True
False	True	False	False	True
False	False	True	True	True

notes.arpanadhikari7.com.np

- As [¬(A∨B)]⇔[(¬A)∧(¬B)] is a tautology, the statements are equivalent.

- **Entailment:**
  - Entailment means that one thing follows from another:
    - E.g ., x + y =4 entails 4=x + y
    - KB as a haystack and α as a needle. Entailment is like the needle being in the haystack; inference is like finding it.
  - Entailment is a relationship between sentences that is based on semantics
  - Knowledge base KB entails sentence α if and only if α is true in all worlds where KB is true . : KB |= α
  - If an inference algorithm i can derive α from KB, we write: $KB \vdash_i \alpha$
  - which is pronounced "α is derived from KB by i" or "i derives α from KB."

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

- **Soundness:**
  - An inference algorithm that derives only entailed sentences is called sound or truth preserving. Soundness is a highly desirable property.
  - An unsound inference procedure essentially makes things up as it goes along—it announces the discovery of nonexistent needles.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

- **Completeness:**
  - An inference algorithm is complete if it can definitely derive any sentence that is entailed.
  - For real haystacks, which are finite in extent, it seems obvious that a systematic examination can always decide whether the needle is in the haystack.
  - For many knowledge bases, however, the haystack of consequences is infinite, and completeness becomes an important issue.
  - Fortunately, there are complete inference procedures for logics that are sufficiently expressive to handle many knowledge bases.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Exercise

Find Truth Table of following :

- A => (B => A)
- (A => B) => A
- A => ((B ^ C) V ¬ A)
- $((A \wedge B) \rightarrow C) \Leftrightarrow (A \rightarrow (B \rightarrow C))$.

notes.arpanadhikari7.com.np

Inference Methods

- The process by which a conclusion is drawn from given premises.
- But logic is concerned with: does the truth of the conclusion follow from that of the premises?
- Several basic methods for determining whether a given set of premises propositionally entails a given conclusion.
  - TruthTableMethod(EnumerationMethod)
  - Deductive(Proof)Systems
  - Resolution

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

- Enumeration Method:
  - Example1:
  - Let, conclusion $\alpha = A \vee B$ and $KB = (A \vee C) \wedge (B \vee \neg C)$
  - Does *KB* entail *a*?
    - check all possible models; *a* must be true whenever *KB* is true I.e., KB $\longrightarrow$ a is valid.

A	*B*	*C*	$A \vee C$	$(B \vee \neg C)$	*KB*	α
True	*True*	*True*	*True*	*True*	*True*	*True*
True	*True*	*False*	*True*	*True*	*True*	*True*
True	*False*	*True*	*True*	*False*	*False*	*True*
True	*False*	*False*	*True*	*True*	*True*	*True*
False	*True*	*True*	*True*	*True*	*True*	*True*
False	*True*	*False*	*False*	*True*	*False*	*True*
False	*False*	*True*	*True*	*False*	*False*	*False*
False	*False*	*False*	*False*	*True*	*False*	*False*

So, conclusion

KB entails α

Available on notes.arpanadhikari7.com.np

- Example2:
  - Symbols:
    - P is “It is hot”,
    - Q is “It is humid” and
    - R is “It is raining”.
  - KB:
    - P^Q=>R (“If it is hot and humid, then it is raining”),
    - Q=>P (“If it is humid, then it is hot”),
    - Q (“It is humid”).
  - Question:
    - Is it raining? (i.e., is R entailed by KB?)

P, Q, R	$P \wedge Q \Rightarrow R$	$Q \Rightarrow P$	Q	KB	R	$KB \Rightarrow R$
T, T, T	T	T	T	T	T	T
T, T, F	F	T	T	F	F	T
T, F, T	T	T	F	F	T	T
T, F, F	T	T	F	F	F	T
F, T, T	T	F	T	F	T	T
F, T, F	T	F	T	F	F	T
F, F, T	T	T	F	F	T	T
F, F, F	T	T	F	F	F	T

  - So, R is entailed by the KB and we can conclude it is raining.

Available on notes.arpanadhikari7.com.np

Deductive (proof) statements

Done by applying rules of inference directly to the sentences in our knowledge base to construct a proof of the desired sentence without consulting models.

– Example:

- All men are mortal.
- Ram is a man.
- Therefore,Ram is mortal.

notes.arpanadhikari7.com.np

If the number of models is large but the length of the proof is short, then Deductive proof can be more efficient than model checking.

Available on notes.arpanadhikari7.com.np

Important Logical Equivalences

$\sim(\sim p) \equiv p$ **the Double Negative Law**

$\sim(p \vee q) \equiv (\sim p) \wedge (\sim q)$

$\sim(p \wedge q) \equiv (\sim p) \vee (\sim q)$ **De Morgan's Laws**

$p \wedge (q \vee r) \equiv (p \wedge q) \vee (p \wedge r)$

$p \vee (q \wedge r) \equiv (p \vee q) \wedge (p \vee r)$ **the Distributive Laws**

notes.arpanadhikari7.com.np

Rule	Tautology	Name
$p \to q$ p $\therefore q$	$((p \to q) \wedge p) \Rightarrow q$	Modus Ponens (Law of Detachment)
$p \to q$ $\neg q$ $\therefore \neg p$	$((p \to q) \wedge \neg q) \Rightarrow \neg q$	Modus Tollens
$p \to q$ $q \to r$ $\therefore p \to r$	$((p \to q) \wedge (q \to r)) \Rightarrow (p \to r)$	Hypothetical Syllogism (Transitivity)
$p \vee q$ $\neg p$ $\therefore q$	$((p \vee q) \wedge \neg p) \Rightarrow q$	Disjunctive Syllogism
p $\therefore p \vee q$	$p \Rightarrow p \vee q$	Addition
$p \wedge q$ $\therefore p$	$(p \wedge q) \Rightarrow p$	Simplification
p q $\therefore p \wedge q$	$(p) \wedge (q) \Rightarrow (p \wedge q)$	Conjunction
$p \vee q$ $\neg p \vee r$ $\therefore q \vee r$	$((p \vee q) \wedge (\neg p \vee r)) \Rightarrow (q \vee r)$	Resolution

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Let’s look at an example for each of these rules to help us make sense of things.

Let p be “It is raining,” and q be “I will make tea,” and r be “I will read a book.”

Example: Modus Ponens

“If it is raining, then I will make tea.” $p \rightarrow q$
“It is raining.” p
“Therefore, I will make tea.” $\therefore q$

Modus Ponens (Law of Detachment)

$$\big((p \rightarrow q) \wedge p\big) \Rightarrow q$$

notes.arpanadhikari7.com.np

Example — Modus Tollens

"If it is raining, then I will make tea." $p \rightarrow q$
"I do not make tea." $\neg q$
"Therefore, it is not raining." $\therefore \neg p$

Modus Tollens

$$\big((p \rightarrow q) \wedge \neg q\big) \Rightarrow \neg q$$

notes.arpanadhikari7.com.np

Example — Hypothetical Syllogism

"If it is raining, then I will make tea." $p \to q$
"If i make tea, then I will read a book." $q \to r$
"Therefore, if it rains, then I will read a book." $\therefore p \to r$

Hypothetical Syllogism

$$\big((p \to q) \wedge (q \to r)\big) \Rightarrow (p \to r)$$

notes.arpanadhikari7.com.np

Example — Disjunctive Syllogism

"I will make tea, or I will read a book." $q \vee r$
"I will not make tea." $\neg q$
"Therefore, I will read a book." $\therefore r$

Disjunctive Syllogism

$$\big((q \vee r) \wedge \neg q\big) \Rightarrow r$$

notes.arpanadhikari7.com.np

Example — Addition

"I will make tea."

"Therefore, I will make tea, or I will read a book."

q

$\therefore q \vee r$

Addition

$q \Rightarrow q \vee r$

notes.arpanadhikari7.com.np

Simplification

"I will make tea and I will read a book."
"Therefore, if it rains, then i will read a book."

$q \wedge r$
$\therefore q$

Simplification

$(p \wedge q) \Rightarrow p$

notes.arpanadhikari7.com.np

Example- Resolution

"It is raining, or I will make tea." $p \vee q$
"It is not raining, or I will read a book." $\neg p \vee r$
"Therefore, I will make tea, or I will read a book." $\therefore q \vee r$

Resolution

$$\left(\left(p \vee q\right) \wedge \left(\neg p \vee r\right)\right) \Rightarrow \left(q \vee r\right)$$

notes.arpanadhikari7.com.np

Example 1: Construct an argument using rules of inference to show that the hypotheses:

“If it does not rain or if it is not foggy, then the sailing race will be held and the life saving demonstration will go on,” “ If the sailing race is held, then the trophy will be awarded,’ and “The trophy was not awarded” imply the conclusion “ It rained”.

Solution:

Let p = “It rains”, q = “It is foggy”, r = “the sailing race is held”, s = “Life saving demonstration is done”, and t = “ Trophy is awarded”.

Then we have to show that the argument

$[((\neg p \vee \neg q) \rightarrow (r \wedge s)) \wedge (r \rightarrow t) \wedge \neg t] \rightarrow p$ is valid.

notes.arpanadhikari7.com.np

[1] $(r \rightarrow t)$ [Hypothesis]

[2] $\neg t$ [Hypothesis]

[3] $\neg r$ [Modus Tollens using steps 1 and 2]

[4] $((\neg p \vee \neg q) \rightarrow (r \wedge s))$ [Hypothesis]

[5] $\neg(\neg p \vee \neg q) \vee (r \wedge s)$ [Implication of Step 4]

[6] $(p \wedge q) \vee (r \wedge s)$ [De Morgan's Law in Step 5]

[7] $p \vee (r \wedge s)$ [Simplification using step 6]

[8] $p \vee r$ [Simplification using step 7]

Here our original premises changes to $(p \vee r) \wedge \neg r$ [from step 8 and 3]

[9] $r \vee p$ [Commutative law in step 8]

[10] $\neg r$ [From step 3]

[11] p [Disjunctive syllogism using steps 9 and 10]

Hence argument is valid. With conclusion "It rained"

notes.arpanadhikari7.com.np

Example 2: For the set of premises “If I play hockey, then I am sore the next day.” “I use the whirlpool if I am sore.” “ I did not use the whirlpool”. What relevant conclusion can be drawn? Explain the rules of inference used to draw the conclusion.

Solution:

Let p = “I play hockey”, q = “ I am sore”, r = “I use the whirlpool”

Then the above premises are

a) $p \rightarrow q$

b) $q \rightarrow r$

c) $\neg r$

notes.arpanadhikari7.com.np

Using hypothetical syllogism in premises a and b we have $p \rightarrow r$ i.e. “if I play hockey, then I use whirlpool”

Using the modus tollens in premise c and inferred proposition $p \rightarrow r$ we conclude $\neg p$ is true i.e. p is false. p is false means “ I did not play hockey”.

Some Equivalence Laws

Idempotency	$P \vee P = P$ $P \& P = P$
Associativity	$(P \vee Q) \vee R = P \vee (Q \vee R)$ $(P \& Q) \& R = P \& (Q \& R)$
Commutativity	$P \vee Q = Q \vee P$ $P \& Q = Q \& P$ $P \leftrightarrow Q = Q \leftrightarrow P$
Distributivity	$P \& (Q \vee R) = (P \& Q) \vee (P \& R)$ $P \vee (Q \& R) = (P \vee Q) \& (P \vee R)$
De Morgan's laws	$\neg(P \vee Q) = \neg P \& \neg Q$ $\neg(P \& Q) = \neg P \vee \neg Q$
Conditional elimination	$P \rightarrow Q = \neg P \vee Q$
Bi-conditional elimination	$P \leftrightarrow Q = (P \rightarrow Q) \& (Q \rightarrow P)$

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Semantics for Propositional logic

The semantics or meaning of a sentence is just the value true or false. The terms used for semantics of a language are given below.

Valid	A sentence is valid if it is true for all interpretations. Valid sentences are also called tautologies.
Model	An interpretation of a formula or sentence under which the formula is true is called a model of that formula.
Unsatisfiable (contradiction)	It is said to be unsatisfiable if it is false for every interpretation.
Satisfiable	It is said to be satisfiable if it is true for some interpretation.
Equivalence	Two sentences are equivalent if they have the same truth value under every interpretation.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Two Normal (Canonical) Forms

All wffs can be expressed in the following to normal forms

1. CNF (Conjunctive Normal Form)

e.g.: $(A \vee \neg B) \wedge (B \vee \neg D \vee \neg C)$

Clause 1 *clause 2*

notes.arpanadhikari7.com.np

2. DNF (Disjunctive Normal Form)

e.g.: $(A \wedge \neg B) \vee (B \wedge \neg D \wedge \neg C)$

models *models*

Available on notes.arpanadhikari7.com.np

Conversion to CNF

- A sentence that is expressed as a conjunction of disjunctions of literals is said to be in conjunctive normal form (CNF). A sentence in CNF that contains only k literals per clause is said to be in k-CNF.
- *Algorithm:*
  - Eliminate ↔ rewriting $P \leftrightarrow Q$ as $(P \rightarrow Q) \wedge (Q \rightarrow P)$
  - Eliminate → rewriting $P \rightarrow Q$ as $\neg P \vee Q$
  - Use De Morgan's laws to push ¬ inwards:
    - rewrite $\neg(P \wedge Q)$ as $\neg P \vee \neg Q$
    - rewrite $\neg(P \vee Q)$ as $\neg P \wedge \neg Q$
  - Eliminate double negations: rewrite $\neg\neg P$ *as* P
  - Use the distributive laws to get CNF:
    - rewrite $(P \wedge Q) \vee R$ as $(P \vee R) \wedge (Q \vee R)$
  - Flatten nested clauses:
    - $(P \wedge Q) \wedge R$ as $P \wedge Q \wedge R$
    - $(P \vee Q) \vee R$ as $P \vee Q \vee R$

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

- Let's illustrate the conversion to CNF by using an example.

$B \Leftrightarrow (A \vee C)$

- Eliminate $\Leftrightarrow$, replacing $\alpha \Leftrightarrow ß$ with $(\alpha \Rightarrow ß) \wedge (ß \Rightarrow \alpha)$.
  - $(B \Rightarrow (A \vee C)) \wedge ((A \vee C) \Rightarrow B)$
- Eliminate $\Rightarrow$, replacing $\alpha \Rightarrow ß$ with $\neg \alpha \vee ß$.
  - $(\neg B \vee A \vee C) \wedge (\neg (A \vee C) \vee B)$
- Move $\neg$ inwards using de Morgan's rules and double-negation:
  - $(\neg B \vee A \vee C) \wedge ((\neg A \wedge \neg C) \vee B)$
- Apply distributivity law ($\wedge$ over $\vee$) and flatten:
  - $(\neg B \vee A \vee C) \wedge (\neg A \vee B) \wedge (\neg C \vee B)$

Available on notes.arpanadhikari7.com.np

Proof by Resolution

- **Resolution Rules:**
  - **(UR) Unit Resolution:** Unit resolution rule takes a clause (a disjunction of literals) and a literal and produces a new clause. Single literal is also called unit clause.

$$\frac{\ell_1 \vee \cdots \vee \ell_k, \qquad m}{\ell_1 \vee \cdots \vee \ell_{i-1} \vee \ell_{i+1} \vee \cdots \vee \ell_k}$$

  - Where li and m are complementary literals

notes.arpanadhikari7.com.np

  - For Example: $\dfrac{\alpha \vee \beta, \quad \neg\beta}{\alpha}$

Available on notes.arpanadhikari7.com.np

- **General Resolution:** The unit resolution rule can be generalized to the full resolution rule,

$$\frac{\ell_1 \vee \cdots \vee \ell_k, \qquad m_1 \vee \cdots \vee m_n}{\ell_1 \vee \cdots \vee \ell_{i-1} \vee \ell_{i+1} \vee \cdots \vee \ell_k \vee m_1 \vee \cdots \vee m_{j-1} \vee m_{j+1} \vee \cdots \vee m_n}$$

where ℓ_i and m_j are complementary literals.

- **For Example:**

$$\frac{\alpha \vee \beta, \quad \neg\beta \vee \gamma}{\alpha \vee \gamma}$$

Available on notes.arpanadhikari7.com.np

Proof by resolution

- Resolution is used with knowledge bases in CNF (or clausal form), and is complete for propositional logic.

- Resolution takes two clauses and produces a new clause containing all the literals of the two original clauses except the two complementary literals.

- In addition the resulting clause should contain only one copy of each literal. The removal of multiple copies of literals is called **factoring.**

notes.arpanadhikari7.com.np

- For example, if we resolve (A ∨ B) with (A∨¬B), we obtain (A ∨ A), which is reduced to just A.

Available on notes.arpanadhikari7.com.np

- **Resolution Algorithm:**

  - Convert KB into CNF
  - Add negation of sentence to be entailed into KB i.e. $(KB \wedge \neg\alpha)$
  - Then apply resolution rule to resulting clauses.
  - The process continues until:
    - There are no new clauses that can be added
      Hence *KB* **does not** entail α
    - Two clauses resolve to entail the empty clause.
      Hence *KB* **does** entail α

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Resolution Example1

Example: Consider the knowledge base given as: $KB = (B \Leftrightarrow (A \vee C)) \wedge \neg B$
Prove that $\neg A$ can be inferred from above KB by using resolution.

Solution:

At first, convert KB into CNF

$B \Rightarrow (A \vee C)) \wedge ((A \vee C) \Rightarrow B) \wedge \neg B$

$(\neg B \vee A \vee C) \wedge (\neg(A \vee C) \vee B) \wedge \neg B$

$(\neg B \vee A \vee C) \wedge ((\neg A \wedge \neg C) \vee B) \wedge \neg B$

$(\neg B \vee A \vee C) \wedge (\neg A \vee B) \wedge (\neg C \vee B) \wedge \neg B$

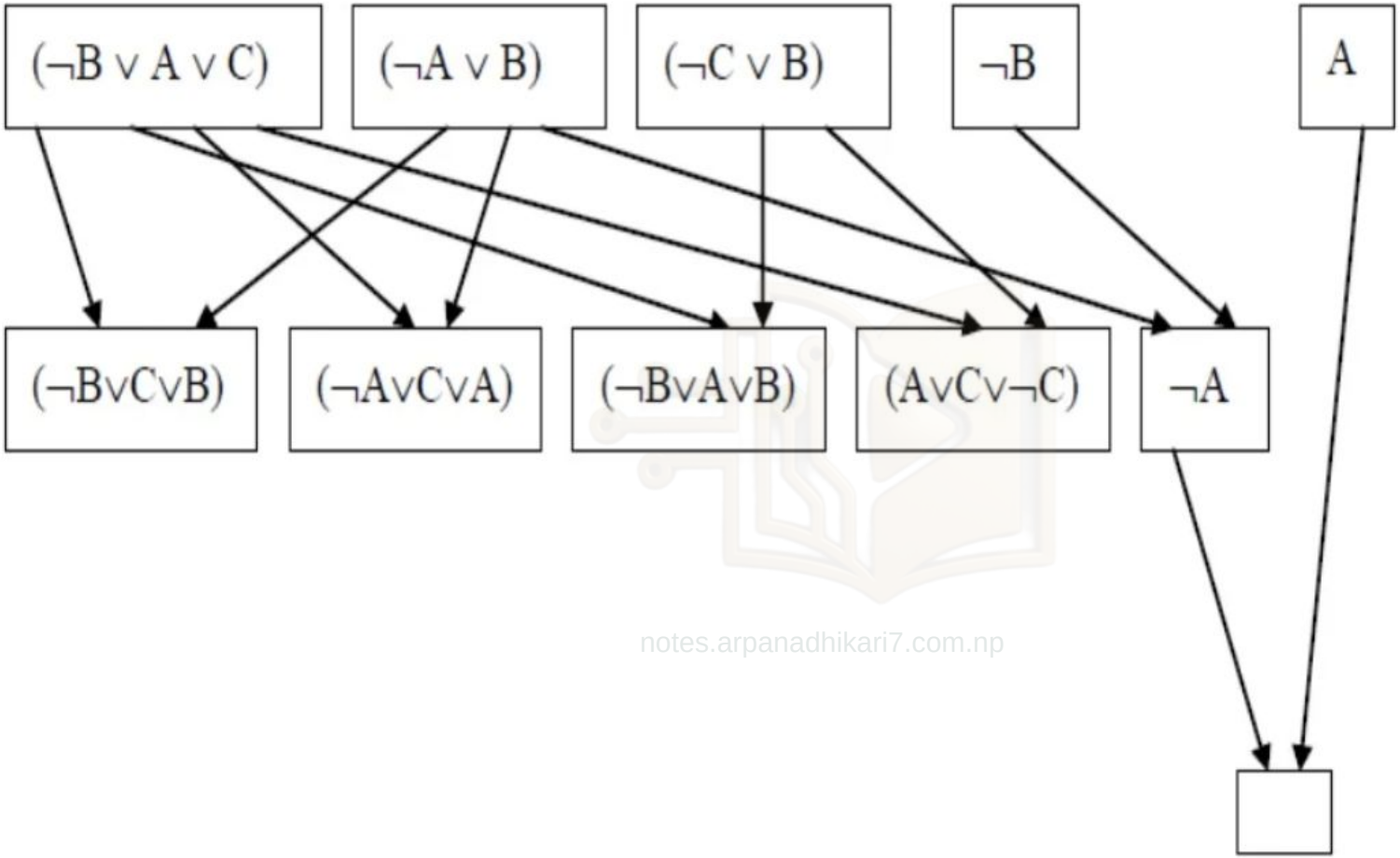
Add negation of sentence to be inferred from KB into KB

Now KB contains following sentences all in CNF
$(\neg B \vee A \vee C)$
$(\neg A \vee B)$
$(\neg C \vee B)$
$\neg B$
A (negation of conclusion to be proved)

Now use Resolution algorithm

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Resolution Example2

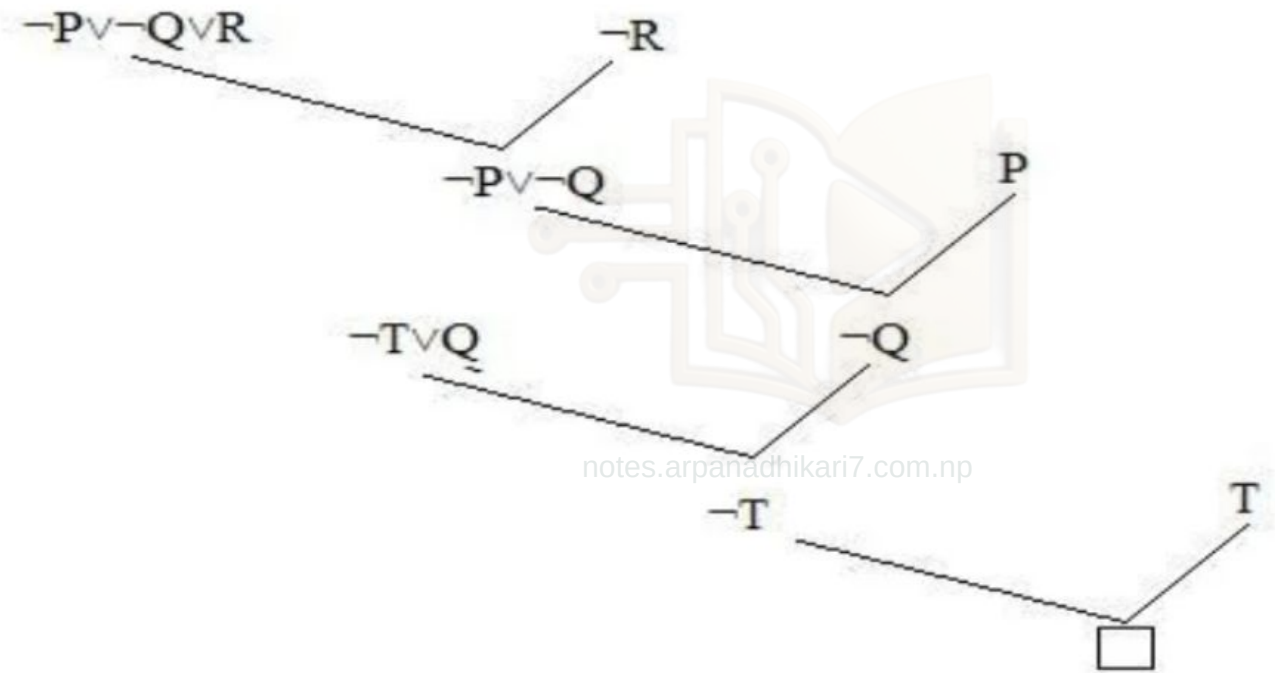
- Suppose we are given the axioms shown in the 2nd column of Table below and we want to prove R.
- First we convert the axioms to clause which is already in clause form.

Sl. No	Given Axioms	Converted to Clause Form
1	P	P
2	$(P \wedge Q) \rightarrow R$	$\neg P \vee Q \vee R$
3	$(S \vee T) \rightarrow Q$	$\neg S \vee Q$
4		$\neg T \vee Q$
5	T	T

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

- Then we begin selecting pairs of clauses to resolve together.



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Pros and cons of propositional logic

- Propositional logic is declarative
- Propositional logic is compositional:
  - meaning of $B \wedge P$ is derived from meaning of B and of P
- Meaning in propositional logic is context-independent
  - (unlike natural language, where meaning depends on context)
- Propositional logic has very limited expressive power (unlike natural language)

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Comparison Between propositional logic and FOPL

- Propositional logic assumes the world contains facts, whereas first-order logic (like natural language) assumes the world contains:
  - Objects: people, houses, numbers, colors, baseball games, wars, …
  - Relations: red, round, prime, brother of, bigger than, part of, comes between,…
  - Functions: father of, best friend, one more than, plus, …
- The primary difference between PL and FOPL is their ontological commitment(**What** exists in the world — TRUTH)
  - PL: facts hold or do not hold.
  - FL : objects with relations between them that hold or do not hold

Available on notes.arpanadhikari7.com.np

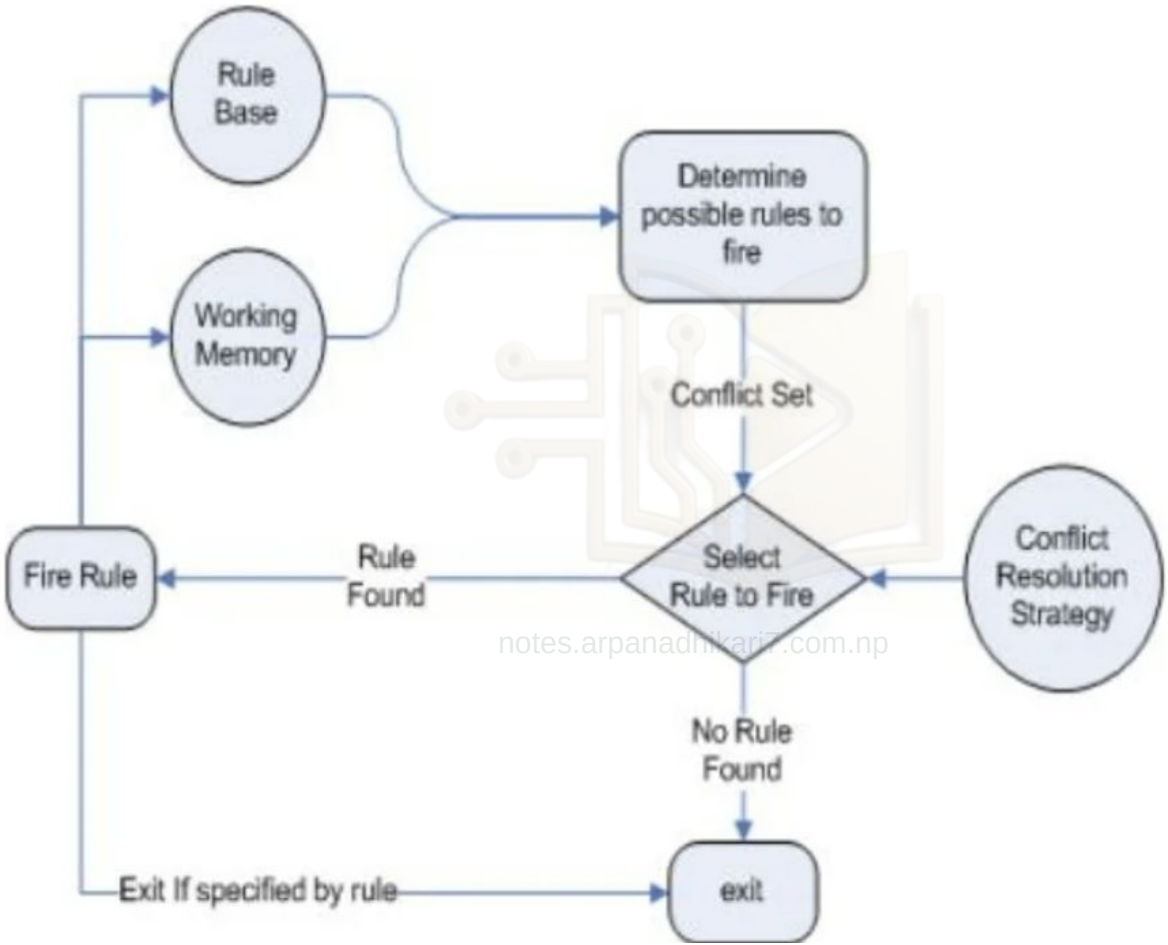
Forward Chaining

- An inference engine using forward chaining searches the inference rules until it finds one where the IF clause is known to be true.
- When found it can conclude, or infer, the THEN clause, resulting in the addition of new information to its dataset. In other words, it starts with some facts and applies rules to find all possible conclusions. Therefore, it is also known as Data Driven Approach.
- The standard definition of a forward-chaining system is that the system operates by repeating the following sequence of operations

1. Examine the rules to find one who's If part is satisfied by the current contents of Working Memory.
2. Fire the rule by adding to Working Memory the facts that are specified in the rules Then part. (The Then part may perform other actions as well, but that can be ignored for now.) This control cycle continues until no rules have satisfied If parts.

Available on notes.arpanadhikari7.com.np

Flowchart for forward chaining



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Backward chaining

- An inference engine using backward chaining would search the inference rules until it finds one which has a THEN clause that matches a desired goal. If the IF clause of that inference rule is not known to be true, then it is added to the list of goals (in order for goal to be confirmed it must also provide data that confirms this new rule) . In other words, this approach starts with the desired conclusion and works backward to find supporting facts. Therefore, it is also known as Goal-Driven Approach.
- Backward-chaining systems try to satisfy the goals in the goal stack. They do this by finding rules that can conclude the information needed by the goal, and trying to make the If parts of those rules satisfied . In more detail, the standard backward-chaining control cycle is shown in figure.

Available on notes.arpanadhikari7.com.np

Steps for backward chaining

1. Check the conclusions of the rules to find all rules that can satisfy the top goal on the stack. 2. Process these rules one at a time:

a. Evaluate the conditions in the rules If part one at a time:

i. If the condition is currently unknown (that is, if there is not enough information currently known to determine whether the condition is satisfied) push a goal to make that condition known, and recursively invoke the system.

ii. If the condition is known to be unsatisfied, continue with the loop at Step 2.

iii. If it was not possible to determine whether the condition was satisfied, continue with the loop at Step 2.

notes.arpanadhikari7.com.np

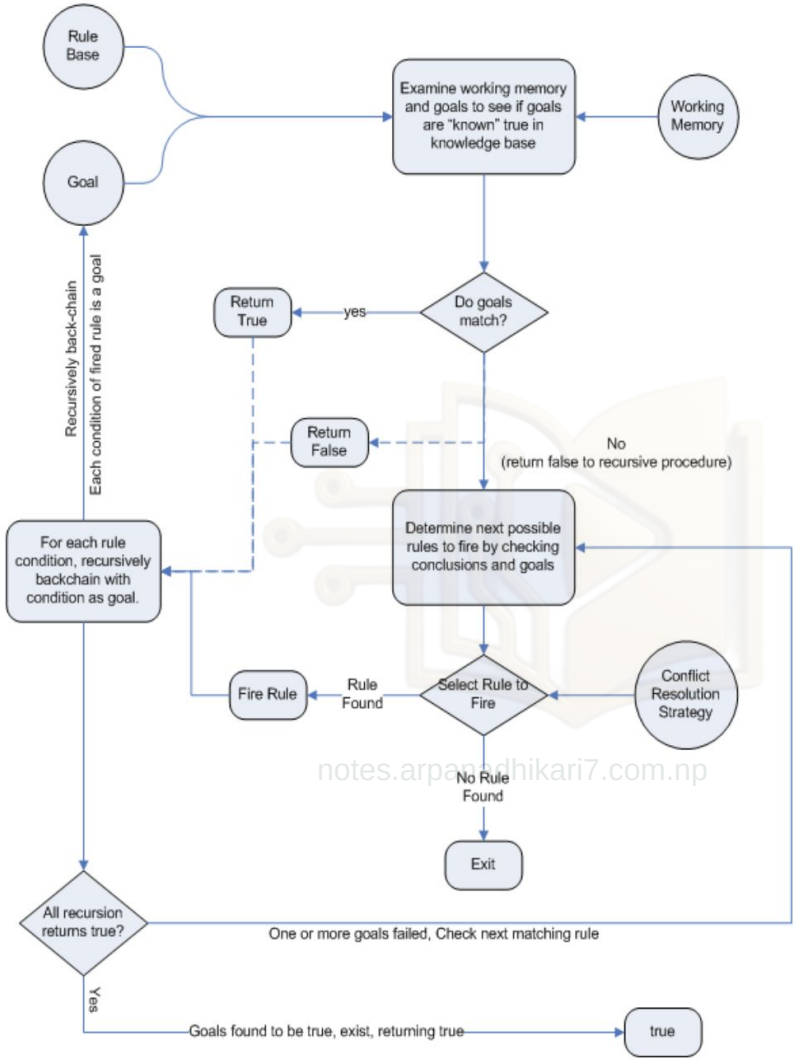
Available on notes.arpanadhikari7.com.np

Steps for backward chaining

b. If all the conditions in the selected rule are satisfied, add to Working Memory the facts specified in the Then part of the rule, pop the goal off the stack, and return from this invocation of the system. The system terminates with success when the goal stack is empty. It terminates with failure if the system runs out of rules to try in Step 2.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Predicate Logic

- propositional logic is best to illustrate the basic concepts of logic and knowledge-based agents.
- But, Propositional logic is limited in several ways.
  - Hard to represent information concisely.
  - Must deal with facts that are either TRUE or FALSE.
- **Predicate Logic!** a more powerful logic (use foundation of propositional logic) by adding more expressive concepts.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Predicates

A Predicate is a declarative sentence whose true/false value depends on one or more variables.

The statement “x is greater than 3” has two parts:

- the subject: x is the subject of the statement
- the predicate: “is greater than 3” (a property that the subject can have) .

We denote the statement “x is greater than 3” by P (x), where P is the predicate “is greater than 3” and x is the variable.

The statement P(x) is also called the value of propositional function P at x.

Available on notes.arpanadhikari7.com.np

Assign a value to x, so P(x) becomes a proposition and has a truth value:

- P (5) is the statement “5 is greater than 3”, so P (5) is true.
- P (2) is the statement “2 is greater than 3”, so P (2) is false.

Prime(x) = “x is a prime number.”

- Prime(2) is true, since the only numbers that divide 2 are 1 and itself.
- Prime(9) is false, since 3 divides 9.

C(x, y)=“x is the capital of y”.

- C(Ottawa,Canada) is true.
- C(Buenos Aires,Brazil) is false.

E(x, y, z) = “x + y = z”.

- E(2, 3, 5) is ...
- E(4, 4, 17) is ...

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

- First-order logic (FOL) models the world in terms of
  - **Objects,** which are things with individual identities
  - **Properties** of objects that distinguish them from others
  - **Relations** that hold among sets of objects
  - **Functions,** which are a subset of relations where there is only one "value" for any given "input"
- Examples:
  - Objects: Students, lectures, companies, cars ...
  - Properties: blue, oval, even, large, ...
  - Relations: Brother, bigger-than, outside, part-of, has-color, occurs-after, owns, visits, precedes, ...
  - Functions: father-of, best-friend, second-half, one more-than ...

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Representing knowledge in FOPL

- The basic syntactic elements of first order logics are the symbols.
- Formula in FOPL contains two types of Symbols. They are:

1. User defined symbols

- Constants:
  - 3, John
  - Individuals
- Functions:
  - f,g,h
  - mappings
- Predicates:
  - P(x,y)
  - functions whose range is {True,False}

2. Logic defined symbols

- Variables:
  x,y,z
  Can be instantiated
- Logical Operators()
- Truth Symbols (TRUE, FALSE)

Quantifiers:

- ∀ for all
- ∃ there exists

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

User defined symbols

- Constant Symbols :
  - The objects from the real world are represented by constant symbols.
  - For instance, the symbol "Ram" may represent a certain individual called Ram.
  - E.g., Hari, Book, Three, etc.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

- **Function Symbols:**
  - Functions are a subset of the relations in which there is only one ``value'' for any given ``input''.
  - i.e., Function symbols denote mappings from elements of a domain to elements of a domain.
  - E.g., FatherOf(Ram), Cosine(0), etc.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

- Predicate Symbols:

  - Properties of objects are represented by predicates applied to those objects (P(a), ...): e.g "male(Ram)" represents that Ram is a male.
  - Relationships between objects are represented by predicates with more arguments: e.g., "Brother(Ram, Hari)" represents the fact that Ram is the brother of Hari.
  - The value of a predicate is one of the Boolean constants T (i.e. true) or F (i.e. false)."brother(Ram, Hari) = T" means that the sentence "Ram is the brother of Hari" is true. "Brother(Tom, Bob) = F" means that the sentence "Ram is the Brother of Hari" is false.
  - Besides constants, the arguments of the predicates may be functions (f, g,...) or variables (x, y,...).
  - E.g. likes(john, mary), valuable(gold), Brother(father(Ram), Krishna), greter-than(x, y) etc.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Logic defined symbols

- First order logic provides some symbols on its own:
  - Truth Symbols : that are TRUE and FALSE.
  - Quantifiers:
    - Universal Quantifier (∀): means "for all".
    - Existential quantification (∃): means "there exists".
  - Logical Operators: order of précédence is left to right but the quantifiers have the same precedence as NOT.
    - ( ), ¬, ∀ , ∃, ∧, ∨, => , <=>

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

- **Variable Symbols:**
  - Variable symbols represent potentially any element of a domain and allow the formulation of general statements about the elements of the domain.
  - E.g., x, y, z,…. Etc.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Sentences in first order logic

- Sentences in FOPL are built from terms and predicates:
  - A Term is an expression referring to an object; Therefore Terms can be:
    - constant symbols,
    - function symbols and
    - variable symbols .
    - A term with no variables is a **ground term** (
      - » E.g., john, father_of(john) and etc.
  - Predicate Symbols for referring to relations
    - e.g., green(grass)

Available on notes.arpanadhikari7.com.np

Sentences in first order logic

- A sentence in FOL is either an atomic sentence or complex sentence.
- Atom (Atomic Sentence):
  - an n-ary predicate symbols that has value TRUE or FALSE.
  - Each of the n-ary arguments are terms.
  - predicate ($term_1$, $term_2$, …, $term_k$)

  where , term = function($term_1$, $term_2$, …, $term_k$) or constant, or variable

E,g., father(Ram, Hari), greater_than(x,y) and etc…

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Sentences in first order logic

- Complex Sentence:
  - A complex sentence is defined as a bunch of atomic sentences joined together using logical connectives (like AND, OR, IMPLICATION, BICONDITIONAL, NOT).
    - » ¬S,
    - » S1 ∧ S2,
    - » S1 ∨ S2,
    - » S1 ⇒ S2,
    - » S1 ⇔ S2
    - » Quantifier <Variable> <Sentence>

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Predicate Logic Syntax

Sentence	→	*AtomicSentence*
	|	*(Sentence* `Connective` *Sentence)*
	|	`Quantifier` *Variable, ... Sentence*
	|	¬ *Sentence*
AtomicSentence	→	*Predicate(Term, ...)* | *Term = Term*
Term	→	*Function(Term, ...)* | *Constant* | *Variable*
Connective	→	∧ | ∨ | ⇒ | ⇔
Quantifier	→	∀ | ∃
Constant	→	*A, B, C, X_1, X_2, Jim, Jack*
Variable	→	*a, b, c, x_1, x_2, counter, position, ...*
Predicate	→	*Adjacent-To, Younger-Than, HasColor, ...*
Function	→	*Father-Of, Square-Position, Sqrt, Cosine*

ambiguities are resolved through precedence or parentheses

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Quantifiers in first order logic

- FOPL also provides variable quantifiers that allow the expression of properties for entire collections of objects instead of enumerating objects by name.
- Two types of quantifiers:
  - Universal quantifier (∀): means "for all".
  - Existential quantification (∃): means "there exists".
- Quantifiers are used with sentences containing variable symbols.
  - Let X be a variable symbol and P be a sentence.

    Then (∀ X, P(X)) is a sentence and (∃ X, P(X)) is a sentence.

Available on notes.arpanadhikari7.com.np

- **Universal quantification:**
  - Often associated with English words "all", "everyone", "always", etc.
  - Syntax: ∀<Variables> <sentence>
  - E.g., Everyone at CAB is smart:

    $\forall x\ At(x, CAB) \Rightarrow Smart(x)$

    (we can also read this as "if X is at CAB, then X is smart)

- **∀x P(x) is true in a model M iff P(x) is true for all x in the model**
  - Roughly speaking, equivalent to the conjunction of instantiations of P
  - E.g.,: $At(Ram,CAB) \Rightarrow Smart(Ram) \wedge At(Hari,CAB) \Rightarrow Smart(Hari) \wedge \ldots$

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

- Typically, $\Rightarrow$ is the main connective with $\forall$
  - A universal quantifier is also equivalent to a set of implications over all objects
- Common mistake: using $\wedge$ as the main connective with $\forall$:
  - $\forall x$ At(x, CAB) $\wedge$ Smart(x)
  - Means "Everyone is at CAB and everyone is smart"
  - You rarely use universal quantification to make blanket statements about every individual in the world (because such statement is hardly true)

notes.arpanadhikari7.com.np

e.g., $\forall x\ human(x) \Rightarrow mortal(x)$
says, all humans are mortal
but, $\forall x\ human(x) \wedge mortal(x)$
say, everything is human and mortal

Available on notes.arpanadhikari7.com.np

- **Existential quantification:**
  - Often associated with English words “someone”, “sometimes”, etc.
  - Syntax: ∃<variables> <sentence>
  - Example: Someone at CAB is smart:

$$\exists x \; At(x, CAB) \wedge Smart(x)$$

- **∃x P(x) is true in a model m iff P(x) is true for at least one x in the model**
  - Roughly speaking, equivalent to the disjunction of instantiations of P
  - At(Ram, CAB) ∧ Smart(RAM)∨At(Hari, CAB) ∧ Smart(Hari)∨ ...

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

- Typically, $\wedge$ is the main connective with $\exists$
- Common mistake: using $\Rightarrow$ as the main connective with $\exists$

  – E.g.: $\exists x$ At(x, CAB) $\Rightarrow$ Smart(x) is true even if there is anyone who is not at CAB!

e.g., $\exists x\ bird(x) \wedge \neg flies(x)$

says, there is a bird that does not fly

but, $\exists x\ bird(x) \Rightarrow \neg flies(x)$

is also true for anything that is not a bird

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Quantifier Scope

- The part of a logical expression to which a quantifier is applied is called the scope of this quantifier.
- For example, suppose we want to say
  - "everyone who is alive loves someone"
  - $(\forall x)\ alive(x) \rightarrow (\exists y)\ loves(x,y)$
- Here's how we scope the variables

$$(\forall x)\ alive(x) \rightarrow (\exists y)\ loves(x, y)$$

notes.arpanadhikari7.com.np

Scope of x

Scope of y

Available on notes.arpanadhikari7.com.np

Free vs. Bound Variables

- Definition
  - An occurrence of a variable in a formula is bound iff the occurrence is in the scope of a quantifier employing the variable; otherwise it is free.
- Examples
  - $\forall x.P(x, y)$
  - x is bound
  - y is free

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Nesting and mixing quantifiers

- Switching the order of multiple universal quantifiers does not change the meaning;

$$\forall X, \forall Y\ P(X,Y) \Leftrightarrow \forall Y, \forall X, P(X,Y)$$

- For example, "Brothers are siblings" can be written as

$$\forall x \forall y \text{ Brother } (x, y) \Rightarrow \text{Sibling}(x, y) .$$

- Consecutive quantifiers of the same type can be written as one quantifier with several variables.

$$\forall x, y \text{ Brother}(x, y) \Leftrightarrow \text{Sibling}(y, x) .$$

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

- Switching the order of a universal quantifier and an existential quantifier does change meaning.

$\forall X, \exists Y\ P(X,Y)$ means "everyone loves someone"
$\exists Y, \forall X\ P(X,Y)$ means "someone is loved by everyone"

- The order of quantification is therefore very important.

- Use different variables with nested quantifiers to avoid confusion.

Available on notes.arpanadhikari7.com.np

Connections between All and Exists

- We can relate sentences involving $\forall$ and $\exists$ using **De Morgan's laws**:
  1. $(\forall x)\ \neg P(x) \leftrightarrow \neg(\exists x)\ P(x)$
  2. $\neg(\forall x)\ P(x) \leftrightarrow (\exists x)\ \neg P(x)$
  3. $(\forall x)\ P(x) \leftrightarrow \neg\ (\exists x)\ \neg P(x)$
  4. $(\exists x)\ P(x) \leftrightarrow \neg(\forall x)\ \neg P(x)$
- Examples
  1. All dogs don't like cats ↔ No dogs like cats
  2. Not all dogs dance ↔ There is a dog that doesn't dance
  3. All dogs sleep ↔ There is no dog that doesn't sleep
  4. There is a dog that talks ↔ Not all dogs can't talk

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

- Convert the following to the language of predicate logic.
    - Every apple is either green or yellow
    - No apple is blue
    - If an apple is green then its tasty
    - Every man likes a tasty apple
    - Some people like garlic
    - Fido is a dog and a good dog.
    - All basketball players are tall

Available on notes.arpanadhikari7.com.np

- Every apple is either green or yellow

$$\forall X(apple(X) \Rightarrow green(X) \vee red(X))$$

- No apple is blue

$$\forall X(apple(x) \Rightarrow \neg blue(X))$$

- If an apple is green then its tasty

$$\forall X((apple(X) \wedge green(X)) \Rightarrow tasty(X))$$

- Every man likes a tasty apple

$$\forall X \exists Y(man(X) \wedge tastyApple(Y) \Rightarrow likes(X, Y))$$

- Some people like garlic

$$\exists X(person(X) \wedge likes(X, garlic))$$

notes.arpanadhikari7.com.np

- Fido is a dog and a good dog.

$$dog(fido) \wedge gooddog(fido)$$

- All basketball players are tall

$$\forall X(basketballPlayer(X) \Rightarrow tall(X))$$

Available on notes.arpanadhikari7.com.np

All students are smart.
$\forall x (Student(x) \Rightarrow Smart(x))$

There exists a student.
$\exists x \, Student(x)$.

There exists a smart student.
$\exists x (Student(x) \wedge Smart(x))$

Every student loves some student.
$\forall x (Student(x) \Rightarrow \exists y (Student(y) \wedge Loves(x,y)))$

Every student loves some other student.
$\forall x (Student(x) \Rightarrow \exists y (Student(y) \wedge \neg (x = y) \wedge Loves(x,y)))$

There is a student who is loved by every other student.
$\exists x (Student(x) \wedge \forall y (Student(y) \wedge \neg(x = y) \Rightarrow Loves(y,x)))$

Bill is a student.
Student(Bill)

Bill takes either Analysis or Geometry (but not both)
$Takes(Bill, Analysis) \Leftrightarrow \neg Takes(Bill, Geometry)$

Bill takes Analysis or Geometry (or both).
$Takes(Bill, Analysis) \vee Takes(Bill, Geometry)$

Bill takes Analysis and Geometry.
$Takes(Bill, Analysis) \wedge Takes(Bill, Geometry)$

Bill does not take Analysis.
$\neg Takes(Bill, Analysis)$.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

- Examples of English sentences converted to predicate logic
  - John loves Mary
    - Use the verb as the prediate and the nouns as arguments
      - loves(john, mary)
  - Mary is tall
    - Make a one argument predicate
      - tall(mary)
  - If the car is moving then wear the seat belt
    - Generally translate If X then Y into $X \Rightarrow Y$
      - moving(car) $\Rightarrow$ wear(seat_belt)
  - If Paul is hungry then he eats fruit
      - Hungry(paul) $\Rightarrow$ eats(paul, fruit)
  - All Students study
      - $\forall X$ student(X) $\Rightarrow$ study(X)
  - These is something small and slimey on the floor
      - $\exists X$ ((small(X) $\wedge$ slimey(X) $\wedge$ onfloor(X))

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

$\text{Cat}(\text{herschel}) \wedge \text{Cat}(\text{orfy})$	*Herschel is a cat and Orfy is a cat.*
$\text{Dog}(\text{heidi}) \wedge \text{Dog}(\text{willy})$	*Heidi and Willy are dogs.*
$\text{Loves}(\text{heidi}, \text{gary})$	*Heidi loves Gary.*
$\text{Fed}(\text{gary}, \text{heidi}, 2{:}00)$	*Gary fed Heidi at 2 pm.*
$\neg(\text{Dog}(\text{orfy}) \wedge \text{Dog}(\text{willy}))$	*Not both Orfy and Willy are dogs.*
$\neg(\text{Cat}(\text{willy}) \vee \text{Cat}(\text{heidi}))$	*Neither Willy nor Heidi is a cat.*
$\text{Dog}(\text{willy}) \rightarrow \neg\text{Person}(\text{willy})$	*If Willy is a dog, then he isn't a person.*
$\exists x\ (\text{Pet}(x) \wedge \text{Dog}(x))$	*Some pets are dogs.*
$\exists x\ (\text{Pet}(x) \wedge \text{Cat}(x))$	*Some cats are pets.*
$\forall x\ (\text{Pet}(x) \rightarrow (\text{Dog}(x) \vee \text{Cat}(x)))$	*Every pet is either a dog or a cat.*
$\forall x\ (\text{Dog}(x) \rightarrow \neg\text{Cat}(x))$	*No dog is a cat.*
$\neg\exists x\ (\text{Dog}(x) \wedge \text{Cat}(x))$	*No dog is a cat.*
$\forall x\ (\text{Cat}(x) \rightarrow \text{Larger}(\text{willy}, x))$	*Willy is larger than every cat.*
$\neg\exists x\ \text{Fed}(\text{elly}, \text{willy}, x)$	*Elly has never fed Willy.*

Available on notes.arpanadhikari7.com.np

FOPL: Semantic

- An interpretation is required to give semantics to first-order logic. The interpretation is a non-empty "domain of discourse" (set of objects). The truth of any formula depends on the interpretation. The interpretation provides, for each:
  - constant symbol an object in the domain
  - function symbols a function from domain tuples to the domain
  - predicate symbol a relation over the domain (a set of tuples)
- Then we define:
  - universal quantifier $\forall x P(x)$ is True iff $P(a)$ is True for all assignments of domain elements a to x
  - existential quantifier $\exists x P(x)$ is True iff $P(a)$ is True for at least one assignment of domain element a to x

Available on notes.arpanadhikari7.com.np

Interpretations & Models in FOPL

- Definition: An *interpretation* is a mapping which assigns
  - objects in domain to constants in the language
  - functional relationships in domain to function symbols
  - relations to predicate symbols
  - usual logical relationships to connectives and quantifiers: $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow, \forall, \exists$

- Definition: Models
  - An interpretation M is a model for a set of sentences S, if every sentence in S is true with respect to M .

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Interpretations & Models in FOPL

- Example: $s = \forall x\ N(x) \Rightarrow L(x, f(x))$

  where N, L are predicate symbols, and f a function symbol

  - Interpretation 1
    - domain = positive integers
    - $N(x)$ = "x is a natural number"
    - $L(x,y)$ = "x is less than y"
    - $f(x)$ = "predecessor of x" (i.e., x-1)
    - then s says: "any natural number is a less than its predecessor" (of course this is false, so this interpretation is not a model for s)
  - Interpretation 2
    - domain = all people
    - $N(x)$ = "x is a person"
    - $L(x,y)$ = "x likes y"
    - $f(x)$ = "mother of x"
    - then s says: "everyone likes his/her mother"

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

FOPL Inference

- Three major families of first-order inference algorithms:
  - **Forward chaining**
  - **Backward chaining**
  - **Resolution**

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

- Idea: first-order inference can be done by converting the knowledge base to propositional logic and using propositional inference.
- Conversion can be done by using the following inference rules for quantifiers:
  - Universal instantiation
  - Existential instantiation
- applied to sentences with quantifiers to obtain sentences without quantifiers.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

- Universal instantiation (UI):
  - UI Rule: Substitute ground term (term without variables) for the variables.
  - a universally quantified sentence can be replaced by the set of all possible instantiations.
  - After UI we discard the universally quantified sentence.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

- For example: suppose our knowledge base contains just the sentences
  - $\forall x \; King(x) \wedge Greedy(x) \Rightarrow Evil(x)$ (all greedy kings are evil)
  - King(John)
  - Greedy(John)
  - Brother (Richard, John) .

– Then we apply UI to the first sentence using all possible ground-term substitutions from the vocabulary of the knowledge base (in this case, {x/John} and {x/Richard }).

– We obtain

notes.arpanadhikari7.com.np

  - King(John) ∧ Greedy(John) ⇒ Evil(John)
  - King(Richard) ∧ Greedy(Richard) ⇒ Evil(Richard) ,

– and we discard the universally quantified sentence.

Available on notes.arpanadhikari7.com.np

- Existential instantiation(EI):
  - EI rule: the variable is replaced by a single new constant symbol does not appear elsewhere in the knowledge base.
  - Basically, the existential sentence says there is some object satisfying a condition, and applying the existential instantiation rule just gives a name to that object.
  - So, Existential Instantiation can be applied once, and then the existentially quantified sentence can be discarded.
  - For example, we no longer need ∃ x Kill(x, Ram) once we have added the sentence Kill (Hari , Ram).

Available on notes.arpanadhikari7.com.np

For example, from the sentence

$$\exists x \;\; Crown(x) \wedge OnHead(x, John)$$

we can infer the sentence

$$Crown(C_1) \wedge OnHead(C_1, John)$$

as long as C_1 does not appear elsewhere in the knowledge base.

- After the universal instantiation and existential instantiation, the knowledge base is propositional.
- Therefore, we can apply any of the complete propositional inference algorithms

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Simple Proof Example1

- Suppose we have the following sentences in the KB:
  - Anything that barks is a dog.
  - Fido barks.
  - prove that Fido is a dog.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

- Step 1: State the facts you know in FOL
  - We know anything that barks is a dog. State this fact in FOL:

    a. $\forall x \text{Barks}(x) \Rightarrow \text{Dog}(x)$

    - This says "If x barks then x is a dog".

  - We know that Fido barks. State this fact also in FOL:

    b. Barks(Fido)

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

- Step 2: Remove all quantifiers
  - Apply the Universal Instantiation inference rule to remove the universal quantifier $(\forall)$ in sentence a
  - The result is:

$$\text{c. Barks(Fido)} \Rightarrow \text{Dog(Fido)}$$

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

- Now KB in propositional form is:

Barks(Fido)
Barks(Fido) ⇒ Dog(Fido)

- Step 3: See what inference rules can be applied.
  - Think about what we want to do: Eliminate the implication, leaving the sentence "Dog(Fido)"
  - How can we so this?
  - Use Modus Ponens

D. Dog(Fido)

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Inference example2

- Consider the following:
  - "All bill's dogs are brown. Bill owns fred. Fred is a dog".
  - Can we infer "fred is brown"?

- **TRANSLATE INTO FIRST ORDER LOGIC:**
  - "All bill's dogs are brown" -
    $$\forall X \; DOG(X) \wedge OWNS(bill, X) \Rightarrow BROWN(X)$$
  - "Bill owns fred" - $OWNS(bill, fred)$
  - "Fred is a dog" - $DOG(fred)$

- **APPLY INFERENCING RULES:**
  - AND-INTRODUCTION - $DOG(fred) \wedge OWNS(bill, fred)$
  - UNIVERSAL ELIMINATION -
    $$DOG(fred) \wedge OWNS(bill, fred) \Rightarrow BROWN(fred)$$
  - MODUS PONENS - $BROWN(fred)$

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Generalized Modus ponens

- Combination of AND- introduction, Universal elimination, and modus ponens into one step.
- If there is some substitution θ{e.g., x/John} that makes each of the conjuncts of the premise of the implication identical to sentences already in the knowledge base, then we can assert the conclusion of the implication, after applying θ.
  - KB:
    - $\forall$ x King(x) $\wedge$ Greedy(x) $\Rightarrow$ Evil(x)
    - King(John)
    - Greedy(John)
  - Query:
    - Evil(John)
  - For this query, we can infer Evil(john) from the given KB with the substitution θ ={x/John}

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

RESOLUTION

- **Conjunctive normal form for first-order logic :**
  - As in the propositional case, first-order resolution requires that sentences be in conjunctive normal form (CNF)—that is, a conjunction of clauses, where each clause is a disjunction of literals.
  - Literals can contain variables, which are assumed to be universally quantified.
  - For example:
    - [Animal (F(x)) ∨ Loves(G(x), x)] ∧ [¬Loves(x, F(x)) ∨ Loves(G(x), x)]

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

- **Conversion to CNF**
  - The procedure for conversion to CNF is similar to the propositional case.
  - We illustrate the procedure by translating the sentence "Everyone who loves all animals is loved by someone," or
    - $\forall x [\forall y \text{ Animal}(y) \Rightarrow \text{Loves}(x, y)] \Rightarrow [\exists y \text{ Loves}(y, x)]$.

notes.arpanadhikari7.com.np

Step1: Eliminate Bi-Implications and implications

$\forall x [\neg \forall y \neg \text{Animal}(y) \vee \text{Loves}(x, y)] \vee [\exists y \text{ Loves}(y, x)]$.

Available on notes.arpanadhikari7.com.np

- Step2: Move ¬ inwards
  - In addition to the usual rules for negated connectives, we need rules for negated quantifiers. Thus, we have
    - ¬∀x p becomes ∃ x ¬p
    - ¬∃x p becomes ∀ x ¬p .
  - Our sentence becomes:
    - ∀ x [∃ y ¬(¬Animal(y) ∨ Loves(x, y))] ∨ [∃ y Loves(y, x)] .
    - ∀ x [∃ y ¬¬Animal(y) ∧ ¬Loves(x, y)] ∨ [∃ y Loves(y, x)] .
- Step3: Eliminate double negation (¬¬)
  - ∀ x [∃ y Animal (y) ∧¬Loves(x, y)] ∨ [∃ y Loves(y, x)] .

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

- Step4: Standardize variables (Rename)
  - Rename bound variables so that each only occurs once
  - For sentences like $(\exists x P(x)) \vee (\exists x Q(x))$ which use the same variable name twice, change the name of one of the variables.
    - Thus, we have
    - $\forall x [\exists y \text{ Animal } (y) \wedge \neg \text{Loves}(x, y)] \vee [\exists z \text{ Loves}(z, x)]$.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

- Step 5 : Move quantifiers to the left
  - $(\forall x P(x)) \vee Q$
    - Can be written as: $\forall x(P(x) \vee Q)$ and
  - $(\exists x P(x)) \vee (\exists y Q(y))$
    - Can be written as: $\exists x \exists y (P(x) \vee Q(y))$
  - Thus, we have
  - $\forall x \exists y \exists z[[\text{Animal}(y) \wedge \neg \text{Loves}(x, y)] \vee \text{Loves}(z, x)]]$.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

- Step 6: Skolemize to eliminate existential quantifiers
  - Skolemization is the process of removing existential quantifiers by elimination.
  - If ∃is not in the scope of ∀ then eliminate ∃ and replace existentially quantified variable by a constant not in the knowledge base.
  - E.g., a translate ∃x P(x) into P(A), where A is a new constant.
  - If ∃is in the scope of ∀ then eliminate ∃ and replace existentially quantified variable by a function with argument universally quantified variables in whose scope the existential quantifier appears.
    - ∀ x [Animal (F(x)) ∧¬Loves(x, F(x))] ∨ Loves(G(x), x) .
    - Here F and G are Skolem functions.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

- Step7: Drop universal quantifiers
  - At this point, all remaining variables must be universally quantified. Moreover, the sentence is equivalent to one in which all the universal quantifiers have been moved to the left. We can therefore drop the universal quantifiers:
  - [Animal (F(x)) ∧ ¬Loves(x, F(x))] ∨ Loves(G(x), x) .

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

- Step8: Distribute ∨ over ∧
  - [Animal (F(x)) ∨ Loves(G(x), x)] ∧ [¬Loves(x, F(x)) ∨ Loves(G(x), x)] .
  - This step may also require flattening out nested conjunctions and disjunctions.
  - The sentence is now in CNF and consists of two clauses.
    - [Animal (F(x)) ∨ Loves(G(x), x)]
    - [¬Loves(x, F(x)) ∨ Loves(G(x), x)

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

- **CNF Conversion summarized algorithm:**
  - Above descriptive steps of CNF conversion can be summarized in the following steps

1. Eliminate implications and bi-implications as in propositional case
2. Move negations inward using De Morgan's laws

   plus rewriting $\neg \forall x P$ as $\exists x \neg P$ and $\neg \exists x P$ as $\forall x \neg P$
3. Eliminate double negations
4. Rename bound variables if necessary so each only occurs once

   e.g. $\forall x P(x) \vee \exists x Q(x)$ becomes $\forall x P(x) \vee \exists y Q(y)$
5. Use equivalences to move quantifiers to the left

   e.g. $\forall x P(x) \wedge Q$ becomes $\forall x\ (P(x) \wedge Q)$ where x is not in Q

   e.g. $\forall x P(x) \wedge \exists y Q(y)$ becomes $\forall x \exists y (P(x) \wedge Q(y))$
6. Skolemise (replace each existentially quantified variable by a **new** term)

   $\exists x P(x)$ becomes $P(a0)$ using a Skolem constant $a0$ since $\exists x$ occurs at the outermost level

   $\forall x \exists y P(x, y)$ becomes $P(x, f0(x))$ using a Skolem function $f0$ since $\exists y$ occurs within $\forall x$
7. The formula now has only universal quantifiers and all are at the left of the formula: drop them
8. Use distribution laws to get CNF and then clausal form

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

The resolution inference rule:

- Two clauses, which are assumed to be standardized apart so that they share no variables, can be resolved if they contain complementary literals.
- Propositional literals are complementary if one is the negation of the other; first-order literals are complementary if one unifies with the negation of the other. Thus, we have

notes.arpanadhikari7.com.np

$$\frac{\ell_1 \vee \cdots \vee \ell_k, \qquad m_1 \vee \cdots \vee m_n}{\text{SUBST}(\theta, \ell_1 \vee \cdots \vee \ell_{i-1} \vee \ell_{i+1} \vee \cdots \vee \ell_k \vee m_1 \vee \cdots \vee m_{j-1} \vee m_{j+1} \vee \cdots \vee m_n)}$$

Available on notes.arpanadhikari7.com.np

Resolution Algorithm

- Algorithm:
  - Convert KB into first order logic expressions.
  - Convert knowledge base (FOPL logic expressions) into CNF
  - convert the negation of query into CNF and then add them into KB.
  - Repeatedly apply resolution to clauses or copies of clauses(a copy of a clause is the clause with all variables renamed) until either the empty clause is derived or no more clauses can be derived.
    - If the empty clause is derived, answer = Yes (query follows form knowledge base).
    - Otherwise answer = No (query does not follow from knowledge base)

Available on notes.arpanadhikari7.com.np

Example of resolution refutation

- Example: Consider the following statements:
  - Everyone who loves all animal is loved by some one.
  - Jack is loves all animal
  - Query: Jack is loved by someone.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

- KB in FOPL:
  - $\forall x\ [\forall y\ Animal(y) \Rightarrow Loves(x, y)] \Rightarrow [\exists y\ Loves(y, x)]$.
  - $\forall y\ Animal(y) \Rightarrow Loves(jack, y)$

- Query in FOPL:
  - $\exists y\ Animal(y) \Rightarrow Loves(y, jack)$

notes.arpanadhikari7.com.np

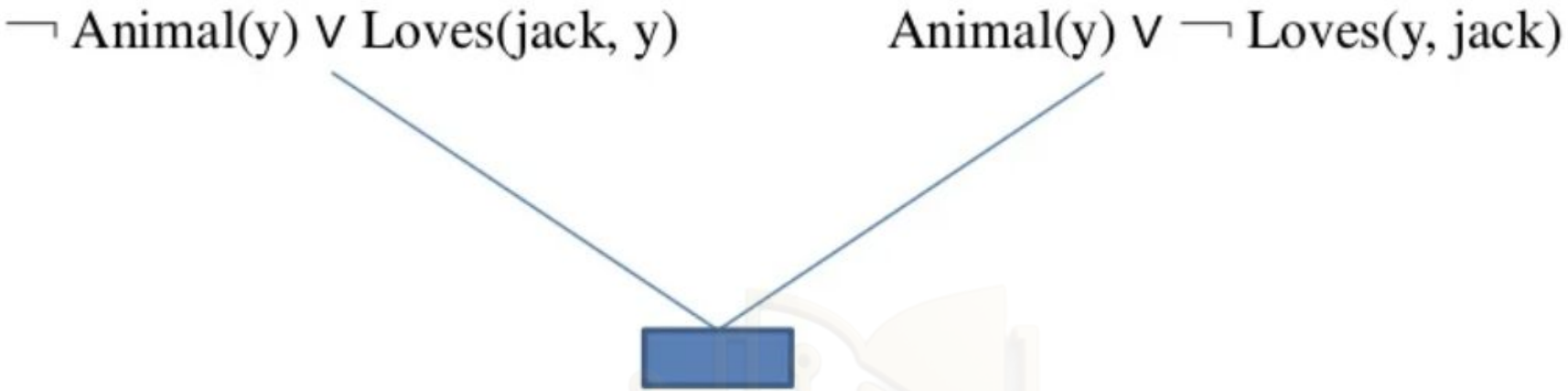
- Negatón of query:
  - $\forall y \neg Animal(y) \Rightarrow \neg Loves(y, jack)$

Available on notes.arpanadhikari7.com.np

- KB and Query in CNF:
    - [Animal (F(x)) ∨ Loves(G(x), x)]
    - [¬Loves(x, F(x)) ∨ Loves(G(x), x)
    - ¬ Animal(y) ∨ Loves(jack, y)
    - Animal(y) ∨ ¬ Loves(y, jack)

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np



Hence jack is loved by someone

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Example of resolution refutation

KB:

1. Anyone passing his history exams and winning the lottery is happy.

2. Anyone who studies or is lucky can pass all his exams.

3. John did not study but John is lucky.

4. Anyone who is lucky wins the lottery.

notes.arpanadhikari7.com.np

query: "Is John happy?".
Use the resolution refutation algorithm to answer the given query.

Available on notes.arpanadhikari7.com.np

(a) Translate the following four English sentences to first order logic (FOL).

1. Anyone passing his history exams and winning the lottery is happy.

2. Anyone who studies or is lucky can pass all his exams.

3. John did not study but John is lucky.

4. Anyone who is lucky wins the lottery.

(b) Convert them to conjunctive normal form (CNF).

(c) Answer the query "Is John happy?". Use the resolution refutation algorithm.

Available on notes.arpanadhikari7.com.np

(a)Translate the following four English sentences to first order logic (FOL).

1. Anyone passing his history exams and winning the lottery is happy.

$$\forall x\ Pass(x, HistoryExam) \wedge Win(x, Lottery) \Rightarrow Happy(x)$$

2. Anyone who studies or is lucky can pass all his exams.

3. John did not study but John is lucky.

4. Anyone who is lucky wins the lottery.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

1. Anyone passing his history exams and winning the lottery is happy.

$$\forall x\ Pass(x, HistoryExam) \wedge Win(x, Lottery) \Rightarrow Happy(x)$$

2. Anyone who studies or is lucky can pass all his exams.

$$\forall x \forall y\ Study(x) \vee Lucky(x) \Rightarrow Pass(x, y)$$

3. John did not study but John is lucky.

4. Anyone who is lucky wins the lottery.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

1. Anyone passing his history exams and winning the lottery is happy.

$$\forall x\ Pass(x, HistoryExam) \wedge Win(x, Lottery) \Rightarrow Happy(x)$$

2. Anyone who studies or is lucky can pass all his exams.

$$\forall x \forall y\ Study(x) \vee Lucky(x) \Rightarrow Pass(x, y)$$

3. John did not study but John is lucky.

$$\neg Study(John) \wedge Lucky(John)$$

4. Anyone who is lucky wins the lottery.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

1. Anyone passing his history exams and winning the lottery is happy.

$$\forall x\ Pass(x, HistoryExam) \wedge Win(x, Lottery) \Rightarrow Happy(x)$$

2. Anyone who studies or is lucky can pass all his exams.

$$\forall x \forall y\ Study(x) \vee Lucky(x) \Rightarrow Pass(x, y)$$

3. John did not study but John is lucky.

$$\neg Study(John) \wedge Lucky(John)$$

4. Anyone who is lucky wins the lottery.

$$\forall x\ Lucky(x) \Rightarrow Win(x, Lottery)$$

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

(b) Convert them to conjunctive normal form (CNF).

1 $\forall x\ Pass(x, HistoryExam) \wedge Win(x, Lottery) \Rightarrow Happy(x)$

2 $\forall x \forall y\ Study(x) \vee Lucky(x) \Rightarrow Pass(x, y)$

3 $\neg Study(John) \wedge Lucky(John)$

4 $\forall x\ Lucky(x) \Rightarrow Win(x, Lottery)$

notes.arpanadhikari7.com.np

First: Implication elimination

Available on notes.arpanadhikari7.com.np

1 $\neg[Pass(x, HistoryExam) \wedge Win(x, Lottery)] \vee Happy(x)$

2 $\neg[Study(x) \vee Lucky(x)] \vee Pass(x, y)$

3 $\neg Study(John) \wedge Lucky(John)$

4 $\neg Lucky(x) \vee Win(x, Lottery)$

notes.arpanadhikari7.com.np

Then: drop the ”for all” quantifiers

Available on notes.arpanadhikari7.com.np

1 $[\neg Pass(x, HistoryExam) \vee \neg Win(x, Lottery)] \vee Happy(x)$

2 $[\neg Study(x) \wedge \neg Lucky(x)] \vee Pass(x, y)$

3 $\neg Study(John) \wedge Lucky(John)$

4 $\neg Lucky(x) \vee Win(x, Lottery)$

notes.arpanadhikari7.com.np

Then: move the negation inwards

Available on notes.arpanadhikari7.com.np

1 $\neg Pass(x, HistoryExam) \vee \neg Win(x, Lottery) \vee Happy(x)$

2 $[\neg Study(x) \vee Pass(x, y)] \wedge [\neg Lucky(x) \vee Pass(x, y)]$

3 $\neg Study(John) \wedge Lucky(John)$

4 $\neg Lucky(x) \vee Win(x, Lottery)$

notes.arpanadhikari7.com.np

Then: distribute the ”ors”

Available on notes.arpanadhikari7.com.np

1 $\neg Pass(x, HistoryExam) \vee \neg Win(x, Lottery) \vee Happy(x)$

2a $\neg Study(x) \vee Pass(x, y)$

2b $\neg Lucky(x) \vee Pass(x, y)$

3a $\neg Study(John)$

3b $Lucky(John)$

4 $\neg Lucky(x) \vee Win(x, Lottery)$

notes.arpanadhikari7.com.np

Then: separate the ”and” sentences into individual sentences

Available on notes.arpanadhikari7.com.np

(c) Query and resolution refutation

1	$\neg Pass(x, HistoryExam) \vee \neg Win(x, Lottery) \vee Happy(x)$
2a	$\neg Study(x) \vee Pass(x, y)$
2b	$\neg Lucky(x) \vee Pass(x, y)$
3a	$\neg Study(John)$
3b	$Lucky(John)$
4	$\neg Lucky(x) \vee Win(x, Lottery)$

Knowledge Base (KB)

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Note: Construct the resolution tree for the clauses given below as in propositional logic but use the predicate resolution rule instead of resolution rule of propositional logic.

1 $\neg Pass(x, HistoryExam) \vee \neg Win(x, Lottery) \vee Happy(x)$

2a $\neg Study(x) \vee Pass(x, y)$

2b $\neg Lucky(x) \vee Pass(x, y)$

3a $\neg Study(John)$

3b $Lucky(John)$

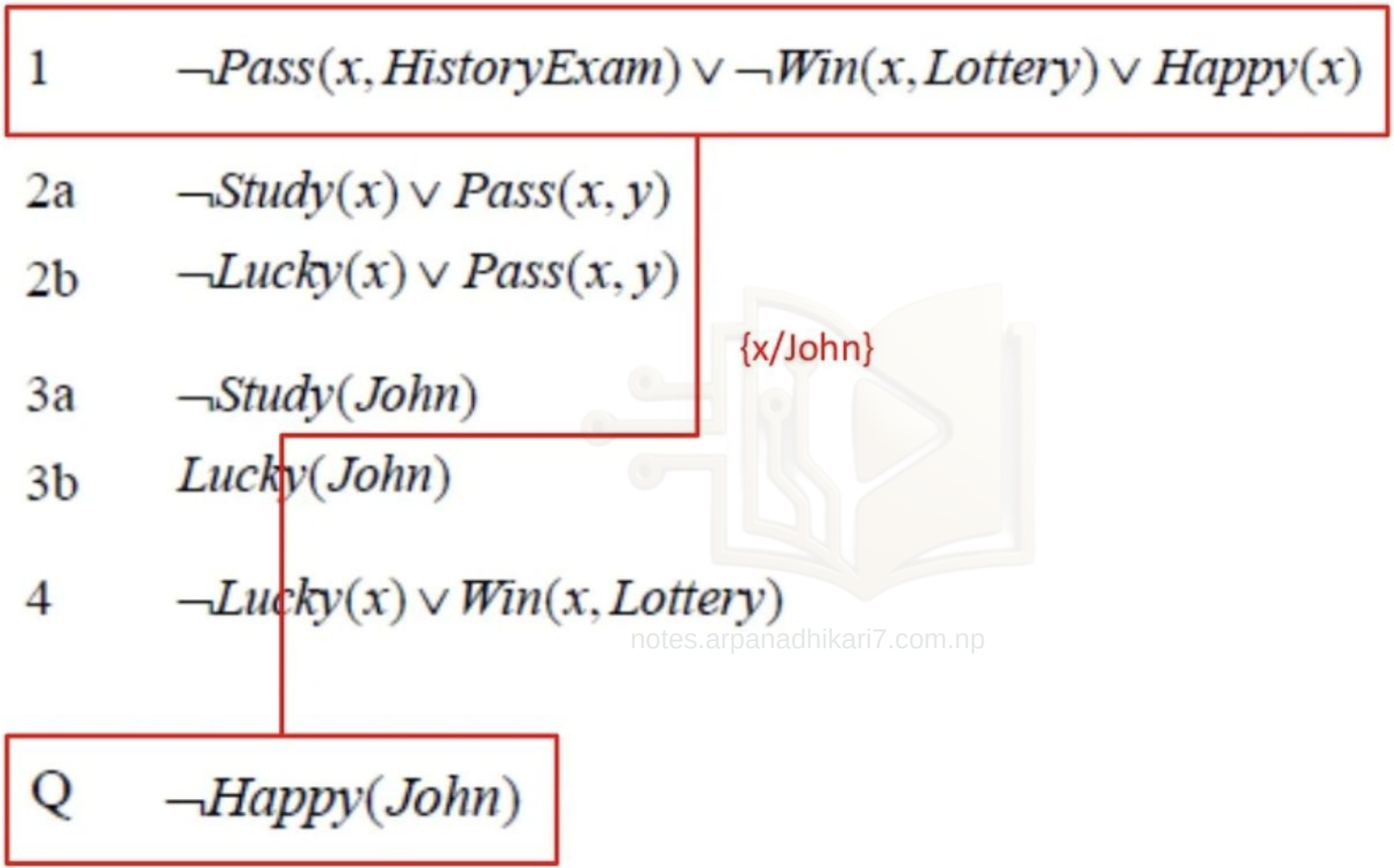
4 $\neg Lucky(x) \vee Win(x, Lottery)$

Q $\neg Happy(John)$

The negation of our query

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

1 $\neg Pass(x, HistoryExam) \vee \neg Win(x, Lottery) \vee Happy(x)$

2a $\neg Study(x) \vee Pass(x, y)$

2b $\neg Lucky(x) \vee Pass(x, y)$

3a $\neg Study(John)$

{x/John; y/HistoryExam}

3b $Lucky(John)$

4 $\neg Lucky(x) \vee Win(x, Lottery)$

notes.arpanadhikari7.com.np

$\neg Pass(John, HistoryExam) \vee \neg Win(John, Lottery)$ 5

Q $\neg Happy(John)$

Available on notes.arpanadhikari7.com.np

1 $\neg Pass(x, HistoryExam) \vee \neg Win(x, Lottery) \vee Happy(x)$

2a $\neg Study(x) \vee Pass(x, y)$

2b $\neg Lucky(x) \vee Pass(x, y)$

3a $\neg Study(John)$

3b $Lucky(John)$

4 $\neg Lucky(x) \vee Win(x, Lottery)$ {x/John}

notes.arpanadhikari7.com.np

$\neg Pass(John, HistoryExam) \vee \neg Win(John, Lottery)$ 5

$\neg Lucky(John) \vee \neg Win(John, Lottery)$ 6

Available on notes.arpanadhikari7.com.np

1 $\neg Pass(x, HistoryExam) \vee \neg Win(x, Lottery) \vee Happy(x)$

2a $\neg Study(x) \vee Pass(x, y)$

$\neg Lucky(John)$ 7

3a $\neg Study(John)$

3b $Lucky(John)$

4 $\neg Lucky(x) \vee Win(x, Lottery)$ {x/John}

$\neg Pass(John, HistoryExam) \vee \neg Win(John, Lottery)$ 5

$\neg Lucky(John) \vee \neg Win(John, Lottery)$ 6

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

1 $\neg Pass(x, HistoryExam) \vee \neg Win(x, Lottery) \vee Happy(x)$

2a $\neg Study(x) \vee Pass(x, y)$

$\neg Lucky(John)$ 7

3a $\neg Study(John)$

3b $Lucky(John)$

$\varnothing$

4 $\neg Lucky(x) \vee Win(x, Lottery)$

notes.arpanadhikari7.com.np

$\neg Pass(John, HistoryExam) \vee \neg Win(John, Lottery)$ 5

$\neg Lucky(John) \vee \neg Win(John, Lottery)$ 6

Available on notes.arpanadhikari7.com.np

Resolution Example2

- In English, the problem is as follows:
  - Everyone who loves all animals is loved by someone.
  - Anyone who kills an animal is loved by no one.
  - Jack loves all animals.
  - Either Jack or Curiosity killed the cat, who is named Tuna.
  - Did Curiosity kill the cat?

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

- First, we express the original sentences, some background knowledge, and the negated goal G in first-order logic:

A. $\forall x\ [\forall y\ Animal(y) \Rightarrow Loves(x,y)] \Rightarrow [\exists y\ Loves(y,x)]$

B. $\forall x\ [\exists z\ Animal(z) \wedge Kills(x,z)] \Rightarrow [\forall y\ \neg Loves(y,x)]$

C. $\forall x\ Animal(x) \Rightarrow Loves(Jack,x)$

D. $Kills(Jack, Tuna) \vee Kills(Curiosity, Tuna)$

E. $Cat(Tuna)$

F. $\forall x\ Cat(x) \Rightarrow Animal(x)$

¬G. $\neg Kills(Curiosity, Tuna)$

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

- Now we apply the CNF conversion procedure to convert each sentence to CNF·

A1. $Animal(F(x)) \vee Loves(G(x), x)$

A2. $\neg Loves(x, F(x)) \vee Loves(G(x), x)$

B. $\neg Loves(y, x) \vee \neg Animal(z) \vee \neg Kills(x, z)$

C. $\neg Animal(x) \vee Loves(Jack, x)$

D. $Kills(Jack, Tuna) \vee Kills(Curiosity, Tuna)$

E. $Cat(Tuna)$

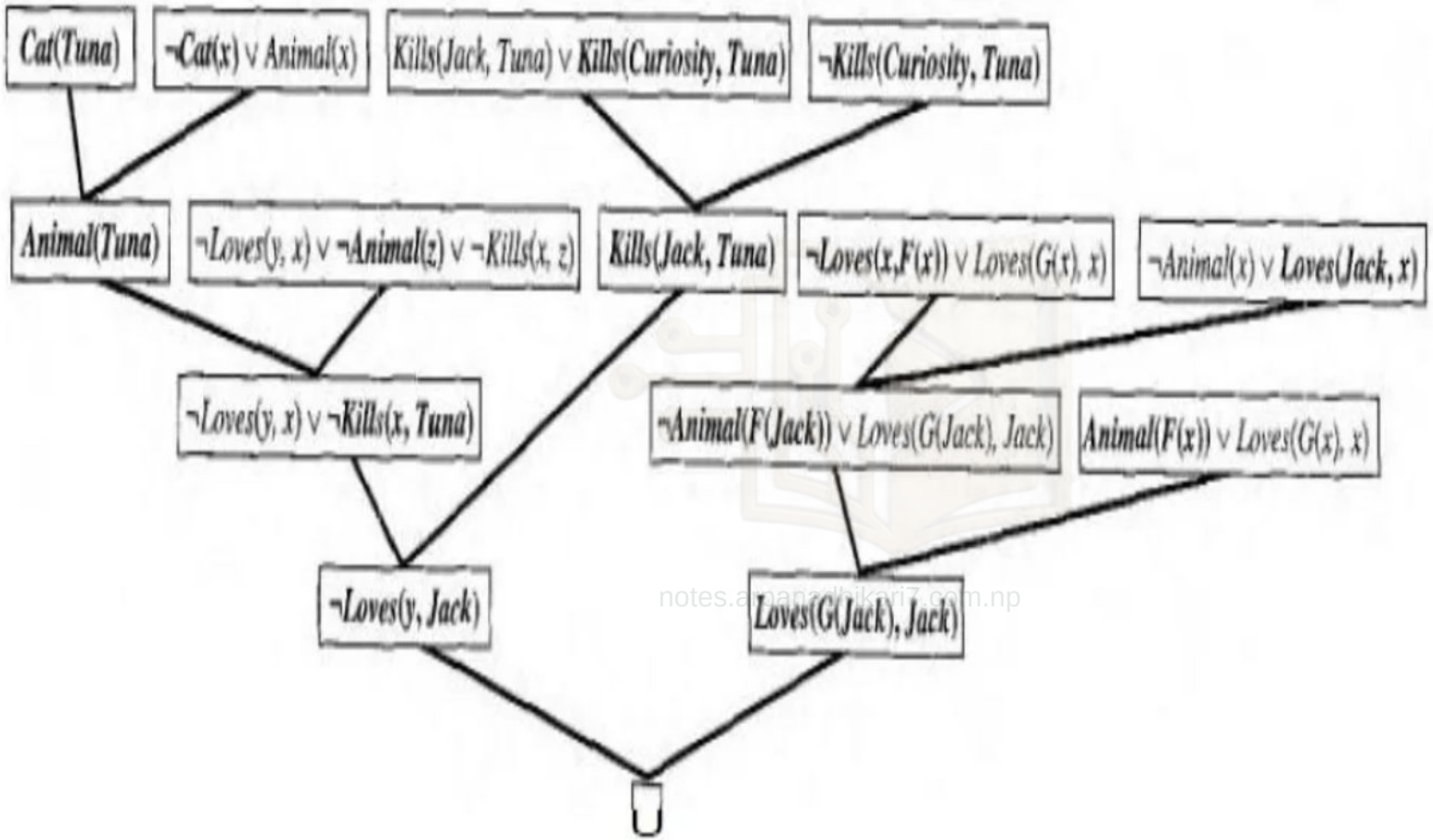
F. $\neg Cat(x) \vee Animal(x)$

¬G. $\neg Kills(Curiosity, Tuna)$

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

- The resolution proof that Curiosity killed the cat is given in Figure below:



Available on notes.arpanadhikari7.com.np

Unification

- Unification is the process of finding substitutions that make different logical expressions look identical.
- takes two sentences and returns a list of substitutions(unifier) to make two sentences match, or failure if no match possible.
- i.e., UNIFY(p, q)=θ , are matched where θ is the list of substitutions in p and q

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

- **Unification rules:**

1. Function symbols and predicate symbols must have identical names and number of arguments.
2. Constant symbols unify with only identical constant symbols.
3. Variables unify with other variable symbols, constant symbols or function symbols
4. Variable symbols may not be unified with other terms in which the variable itself occurs.
   - For example: x can not unify with G(x) since this will lead to G(G(G(….G(x))))

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

- Example: unification

p	q	θ
Knows(John,x)	Knows(John,Jane)	{x/Jane}
Knows(John,x)	Knows(y,OJ)	{x/OJ,y/John}
Knows(John,x)	Knows(y,Mother(y))	{y/John,x/Mother(John)}}
Knows(John,x)	Knows(x,OJ)	{fail}

  - Last unification is failed due to overlap of variables
  - i.e., x can not take the values of John and OJ at the same time.
  - We can avoid this problem(name clashes) by renaming (standardizing apart)
  - E.g., Unify{Knows(John,x) Knows(z,OJ) } = {x/OJ, z/John}

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Semantic Network

- A **semantic net** (or semantic network) is a knowledge representation technique used for propositional information. So it is also called a propositional net.
- Mathematically a semantic net can be defined as a labeled directed graph.
- consist of:
  - nodes,
  - links (edges) and
  - link labels.

notes.arpanadhikari7.com.np

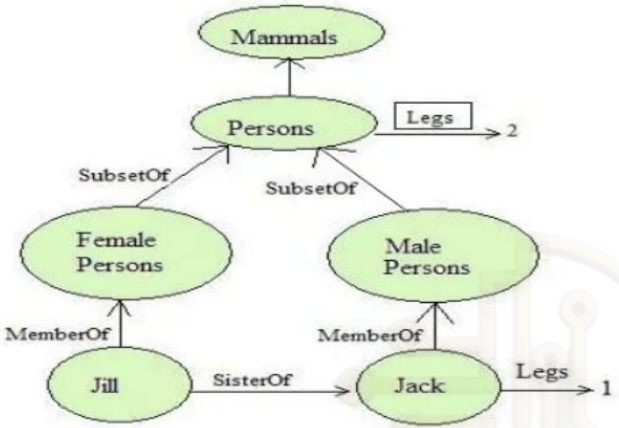
Available on notes.arpanadhikari7.com.np

- Nodes:
  - In the semantic network diagram, nodes appear as circles or ellipses or rectangles to represent objects such as physical objects, concepts or situations.
- Links:
  - appear as arrows to express the relationships between objects, and
- link labels:
  - specify particular relations. Relationships provide the basic structure for organizing knowledge.
  - The objects and relations involved need not be so concrete.
- As nodes are associated with other nodes semantic nets are also referred to as associative nets.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Semantic Network



- In the above figure all the objects are within ovals and connected using labelled arcs.
- Note that there is a link between Jill and FemalePersons with label MemberOf. Simlarly there is a MemberOf link between Jack and MalePersons and SisterOf link between Jill and Jack.
- The MemberOf link between Jill and FemalePersons indicates that Jill belongs to the category of female persons.

Available on notes.arpanadhikari7.com.np

Semantic Network

- **Inheritance Reasoning**
  - Unless there is a specific evidence to the contrary, it is assumed that all members of a class (category) will inherit all the properties of their super classes.
  - So semantic network allows us to perform inheritance reasoning.
    - For example: Jill inherits the property of having two legs as she belongs to the category of FemalePersons which in turn belongs to the category of Persons which has a boxed Legs link with value 2.
  - Semantic nets allows multiple inheritance. So an object can belong to more than one category and a category can be a subset of more than one another category.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Semantic Network

- Inverse Links
  - Semantic network allows a common form of inference known as inverse links.
    - For example: we can have a *HasSister* link which is the inverse of *SisterOf* link.
    - The inverse links make the job of inference algorithms much easier to answer queries such as who the sister of *Jack* is.
    - On discovering that *HasSister* is the inverse of *SisterOf* the inference algorithm can follow that link HasSister from *Jack* to *Jill* and answer the query.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Semantic Network

- Disadvantage Of Semantic Nets
  - One of the drawbacks of semantic network is that the links between the objects represent only binary relations.
    - For example, the sentence Run(Kirtipur Express, Kirtipur, Ratnapark, Today) cannot be asserted directly.
  - There is no standard definition of link names.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Semantic Network

- Advantages Of Semantic Nets
  - Semantic nets have the ability to represent default values for categories.
    - In the above figure Jack has one leg while he is a person and all persons have two legs. So persons have two legs has only default status which can be overridden by a specific value.
  - They convey some meaning in a transparent manner.
  - Semantic nets are simple and easy to understand.

notes.arpanadhikari7.com.np

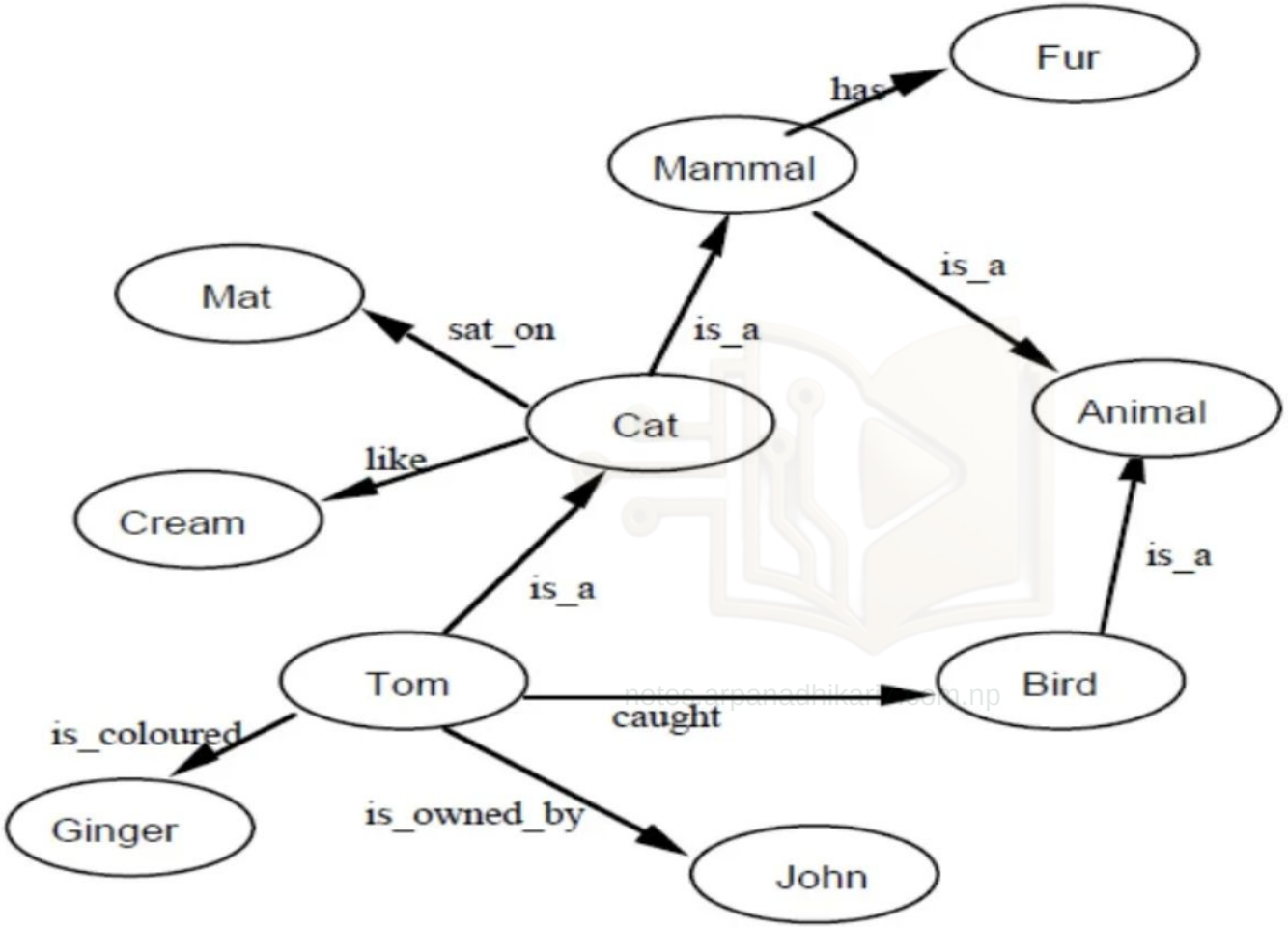
Available on notes.arpanadhikari7.com.np

Example1: semantic network

- Represent the following fact in semantic network
  - Tom is a cat.
  - Tom caught a bird.
  - Tom is owned by John.
  - Tom is ginger in color.
  - Cats like cream.
  - The cat sat on the mat.
  - A cat is a mammal.
  - A bird is an animal.
  - All mammals are animals.
  - Mammals have fur.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np



Available on notes.arpanadhikari7.com.np

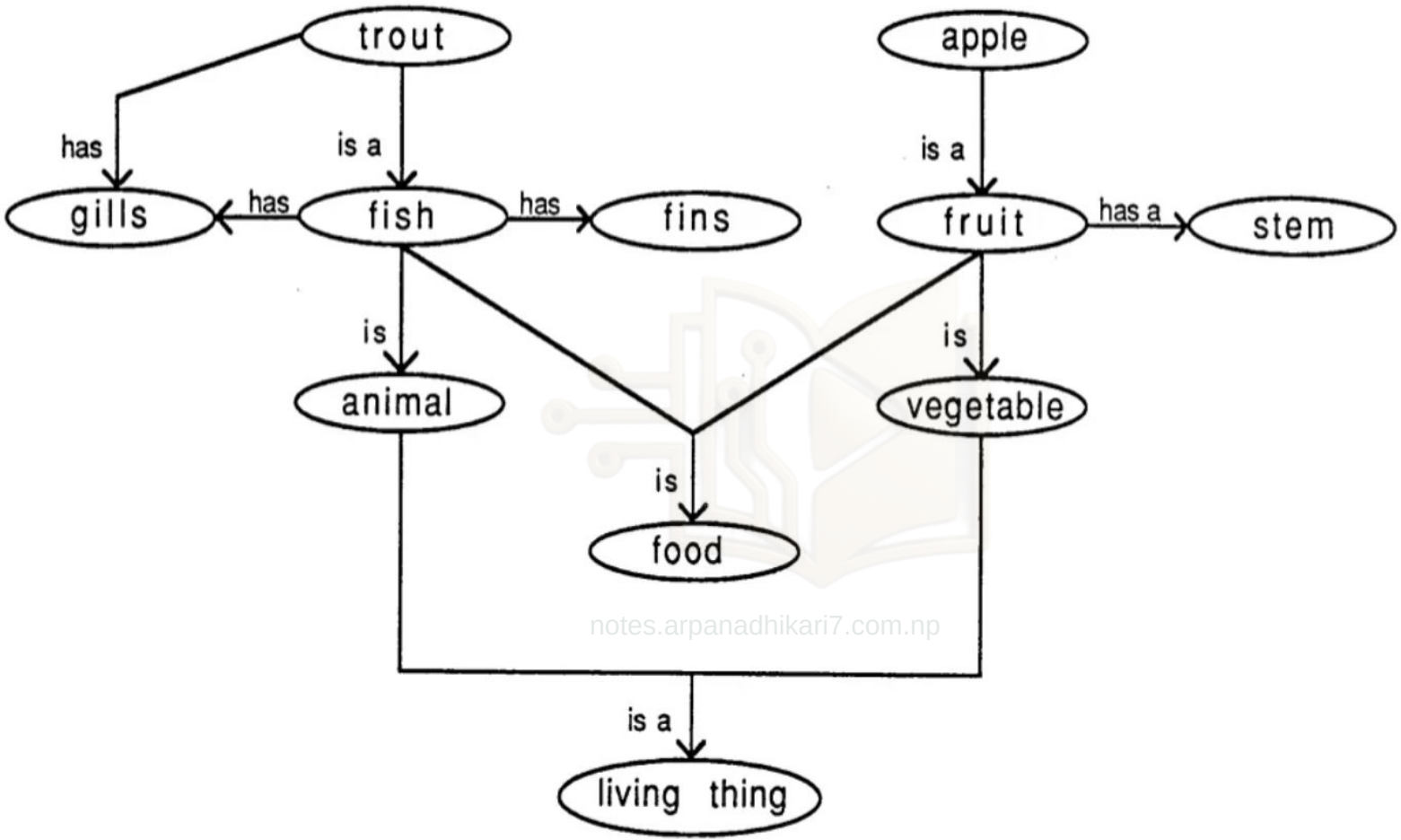
Example2: semantic network

- Represent the following fact in semantic network

(1) A trout is a fish.
(2) A fish has gills.
(3) A fish has fins.
(4) Fish is food.
(5) Fish is animal.
(6) An apple is a fruit.
(7) Fruit has a stem.
(8) Fruit is food.
(9) Fruit is vegetable.
(10) An animal is a living thing.
(11) A vegetable is a living thing.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Frames

A limitation of semantic networks is that additional structure is often necessary to distinguish

– statements about an object's relationships

– properties of the object

A frame is a node with additional structure that facilitates differentiating relationships between objects and properties of objects.

Called a "slot-and-filler" representation

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Rule based Systems

Rule-based systems (RBS) provide automatic problem solving tools for capturing the human expertise and decision making.

RBS are means for codifying the problem solving of human experts.

Experts typically express most of their problem solving techniques in terms of antecedent-consequent rules.

The main properties of rule-based systems are:

- They incorporate practical human knowledge in if-then rules
- Their skill increases proportionally to the enlargement of their knowledge
- They can solve a wide range of potentially complex problems by selecting relevant rules and then combining the results
- They adaptively determine the best sequence of rules to examine
- They explain their conclusions by retracting their lines of reasoning

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Forward Chaining

Forward chaining starts with the available data and uses inference rules to extract more data until a goal is reached.

An inference engine using forward chaining searches the inference rules until it finds one where the antecedent (If clause) is known to be true.

When such a rule is found, the engine can conclude, or infer, the consequent (Then clause), resulting in the addition of new information to its data.

Inference engines will iterate through this process until a goal is reached.

Because the data determines which rules are selected and used, this method is called data-driven.

One of the advantages of forward-chaining over backward-chaining is that the reception of new data can trigger new inferences, which makes the engine better suited to dynamic situations in which conditions are likely to change.

Available on notes.arpanadhikari7.com.np

Example of Forward Chaining

The rule base contains the following four rules: -

If X croaks and eats flies - Then X is a frog

If X chirps and sings - Then X is a canary

If X is a frog - Then X is green

If X is a canary - Then X is yellow

Goal: conclude the color of a pet named Fritz, given that he croaks and eats flies,

Inference Mechanism: - This rule base would be searched and the first rule would be selected, because its antecedent (If Fritz croaks and eats flies) matches our data. Now the consequents (Then X is a frog) is added to the data. - The rule base is again searched and this time the third rule is selected, because its antecedent (If Fritz is a frog) matches our data that was just confirmed. Now the new consequent (Then Fritz is green) is added to our data. - Nothing more can be inferred from this information, but we have now accomplished our goal of determining the color of Fritz.

notes.arpanadhikari7.com.np

Backward Chaining

- Backward chaining systems usually employ a depth-first search strategy.
- Backward chaining starts with a list of goals (or a hypothesis) and works backwards from the consequent to the antecedent to see if there is data available that will support any of these consequents.
- An inference engine using backward chaining would search the inference rules until it finds one which has a consequent (Then clause) that matches a desired goal.
- If the antecedent (If clause) of that rule is not known to be true, then it is added to the list of goals

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Example Backward Chaining

The rule base contains the following four rules: -

If X croaks and eats flies - Then X is a frog

If X chirps and sings - Then X is a canary

If X is a frog - Then X is green

If X is a canary - Then X is yellow

Goal: conclude the color of a pet named Fritz, given that he croaks and eats flies,

Inference Mechanism: - This rule base would be searched and the third and fourth rules would be selected, because their consequents (Then Fritz is green, Then Fritz is yellow) match the goal (to determine Fritz's color). - It is not yet known that Fritz is a frog, so both the antecedents (If Fritz is a frog, If Fritz is a canary) are added to the goal list. - The rule base is again searched and this time the first two rules are selected, because their consequents (Then X is a frog, Then X is a canary) match the new goals that were just added to the list. - The antecedent (If Fritz croaks and eats flies) is known to be true and therefore it can be concluded that Fritz is a frog, and not a canary. - The goal of determining Fritz's color is now achieved (Fritz is green if he is a frog, and yellow if he is a canary, but he is a frog since he croaks and eats flies; therefore, Fritz is green).

Available on notes.arpanadhikari7.com.np

Sources of Uncertainty

- Information is partial.
- Information is not fully reliable.
- Representation language is inherently imprecise.
- Information comes from multiple sources and it is conflicting.
- Information is approximate.
- Non-absolute cause-effect relationships exist.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Uncertainty

Let action A_t = leave for airport t minutes before flight

Will A_t get me there on time?

Problems:

1. partial observability (road state, other drivers' plans, etc.)
2. noisy sensors (traffic reports)
3. uncertainty in action outcomes (flat tire, etc.)
4. immense complexity of modeling and predicting traffic

Hence a purely logical approach either

1. risks falsehood:

   "A_{25} will get me there on time", or

2. leads to conclusions that are too weak for decision making:

   "A_{25} will get me there on time if there's no accident on the bridge and it doesn't rain and my tires remain intact etc. etc."

(A_{1440} might reasonably be said to get me there on time but I'd have to stay overnight in the airport …)

Available on notes.arpanadhikari7.com.np

Methods for handling uncertainty

- Default or nonmonotonic logic: (by default, something is true)
  - Assume my car does not have a flat tire
  - Assume A_{25} works unless contradicted by evidence
- Issues: What assumptions are reasonable? How to handle contradiction?
- Rules with fudge factors:
  - $A_{25} \mid\!\rightarrow_{0.3}$ get there on time
  - *Sprinkler* $\mid\!\rightarrow_{0.99}$ *WetGrass*
  - *WetGrass* $\mid\!\rightarrow_{0.7}$ *Rain*
- Issues: Problems with combination, e.g., *Sprinkler* causes *Rain*??
- Probability
  - Model agent's degree of belief
  - Given the available evidence, A_{25} will get me there on time with probability 0.04

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Basic Probability Theory

- An **experiment** has a set of potential outcomes, e.g., throw a dice
- The **sample space** of an experiment is the set of all possible outcomes, e.g., {1, 2, 3, 4, 5, 6}
- An **event** is a subset of the sample space.
  - {2}
  - {3, 6}
  - even = {2, 4, 6}
  - odd = {1, 3, 5}

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Probabilistic assertions summarize effects of

- laziness: failure to enumerate exceptions, qualifications, etc.
- ignorance: lack of relevant facts, initial conditions, etc.

Subjective probability:

- Probabilities relate propositions to agent's own state of knowledge

  e.g., $P(A_{25} \mid \text{no reported accidents}) = 0.06$

These are not assertions about the world

notes.arpanadhikari7.com.np

Probabilities of propositions change with new evidence:

e.g., $P(A_{25} \mid \text{no reported accidents, 5 a.m.}) = 0.15$

Available on notes.arpanadhikari7.com.np

Probability as Relative Frequency

- An event has a probability.
- Consider a long sequence of experiments. If we look at the number of times a particular event occurs in that sequence, and compare it to the total number of experiments, we can compute a ratio.
- This ratio is one way of estimating the probability of the event.
- P(E) = (# of times E occurred)/(total # of trials)

Available on notes.arpanadhikari7.com.np

Making decisions under uncertainty

Suppose I believe the following:

$P(A_{25}$ gets me there on time $| \ldots) = 0.04$

$P(A_{90}$ gets me there on time $| \ldots) = 0.70$

$P(A_{120}$ gets me there on time $| \ldots) = 0.95$

$P(A_{1440}$ gets me there on time $| \ldots) = 0.9999$

- Which action to choose?

  Depends on my preferences for missing flight vs. time spent waiting, etc.

  - Utility theory is used to represent and infer preferences
  - Decision theory = probability theory + utility theory

Available on notes.arpanadhikari7.com.np

Syntax

- Basic element: random variable
- Similar to propositional logic: possible worlds defined by assignment of values to random variables.
- Boolean random variables

  e.g., *Cavity* (do I have a cavity?)
- Discrete random variables

  e.g., *Weather* is one of <*sunny,rainy,cloudy,snow*>
- Domain values must be exhaustive and mutually exclusive
- Elementary proposition constructed by assignment of a value to a random variable: e.g., *Weather* = *sunny*, *Cavity* = *false* (abbreviated as ¬*cavity*)
- Complex propositions formed from elementary propositions and standard logical connectives e.g., *Weather* = *sunny* ∨ *Cavity* = *false*

Available on notes.arpanadhikari7.com.np

Syntax

- Atomic event: A complete specification of the state of the world about which the agent is uncertain

  e.g., if the world consists of only two Boolean variables *Cavity* and *Toothache*, then there are 4 distinct atomic events:

  $Cavity = false \wedge Toothache = false$

  $Cavity = false \wedge Toothache = true$

  $Cavity = true \wedge Toothache = false$

  $Cavity = true \wedge Toothache = true$

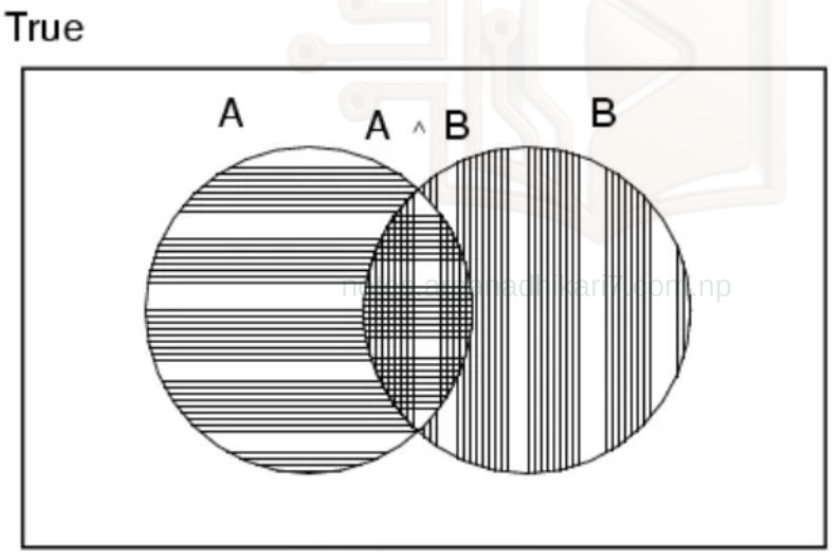
- Atomic events are mutually exclusive and exhaustive

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Axioms of probability

- For any propositions A, B
  - $0 \leq \mathrm{P}(A) \leq 1$
  - $\mathrm{P}(true) = 1$ and $\mathrm{P}(false) = 0$
  - $\mathrm{P}(A \vee B) = \mathrm{P}(A) + \mathrm{P}(B) - \mathrm{P}(A \wedge B)$



Available on notes.arpanadhikari7.com.np

Axioms of Probability Theory

- Suppose P(.) is a probability function, then
  1. for any event E, $0 \leq P(E) \leq 1$.
  2. P(S) = 1, where S is the sample space.
  3. for any two mutually exclusive events E1 and E2,

$$P(E1 \text{ È } E2) = P(E1) + P(E2)$$

- Any function that satisfies the above three axioms is a probability function.

Available on notes.arpanadhikari7.com.np

Prior probability

- Prior or unconditional probabilities of propositions
  e.g., P(*Cavity* = true) = 0.1 and P(*Weather* = sunny) = 0.72 correspond to belief prior to arrival of any (new) evidence

- Probability distribution gives values for all possible assignments:
  P(*Weather*) = <0.72,0.1,0.08,0.1> (normalized, i.e., sums to 1)

- Joint probability distribution for a set of random variables gives the probability of every atomic event on those random variables
  P(*Weather,Cavity*) = a 4 × 2 matrix of values:

Weather =		sunny	rainy	cloudy	snow
Cavity = true		0.144	0.02	0.016	0.02
Cavity = false	0.576	0.08	0.064	0.08	

Every question about a domain can be answered by the joint distribution

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Joint Probability

- Let A, B be two events, the joint probability of both A and B being true is denoted by P(A, B).
- Example:

  P(spade) is the probability of the top card being a spade.

  P(king) is the probability of the top card being a king.

  P(spade, king) is the probability of the top card being both a spade and a king, i.e., the king of spade.

  P(king, spade)=P(spade, king) ???

Available on notes.arpanadhikari7.com.np

Properties of Probability

1. $P(\emptyset E) = 1 - P(E)$
2. If E1 and E2 are logically equivalent, then

   $P(E1)=P(E2)$.

   - E1: Not all philosophers are more than six feet tall.
   - E2: Some philosopher is not more that six feet tall.

     Then $P(E1)=P(E2)$.
3. $P(E1, E2) \leq P(E1)$.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Conditional probability

- Conditional or posterior probabilities

  e.g., P(*cavity* | *toothache*) = 0.8

  i.e., given that *toothache* is all I know

- If we know more, e.g., *cavity* is also given, then we have

  P(*cavity* | *toothache*,*cavity*) = 1

- New evidence may be irrelevant, allowing simplification, e.g.,

  P(*cavity* | *toothache*, *sunny*) = P(*cavity* | *toothache*) = 0.8

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

- Definition of conditional probability:

  $P(a \mid b) = P(a \wedge b) / P(b)$ if $P(b) > 0$

- Product rule gives an alternative formulation:

  $P(a \wedge b) = P(a \mid b)\, P(b) = P(b \mid a)\, P(a)$

- A general version holds for whole distributions, e.g.,

  $\mathbf{P}(\mathit{Weather}, \mathit{Cavity}) = \mathbf{P}(\mathit{Weather} \mid \mathit{Cavity})\, \mathbf{P}(\mathit{Cavity})$

- (View as a set of 4×2 equations, not matrix multiplication)

- Chain rule is derived by successive application of product rule:

$$\begin{aligned}\mathbf{P}(X_1, \ldots, X_n) &= \mathbf{P}(X_1, \ldots, X_{n-1})\, \mathbf{P}(X_n \mid X_1, \ldots, X_{n-1}) \\ &= \mathbf{P}(X_1, \ldots, X_{n-2})\, \mathbf{P}(X_{n-1} \mid X_1, \ldots, X_{n-2})\, \mathbf{P}(X_n \mid X_1, \ldots, X_{n-1}) \\ &= \ldots \\ &= \pi_{i=1}{}^{\wedge}n\, \mathbf{P}(X_i \mid X_1, \ldots, X_{i-1})\end{aligned}$$

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Example

A: the top card of a deck of poker cards is a king of spade

$P(A) = 1/52$

However, if we know

B: the top card is a king

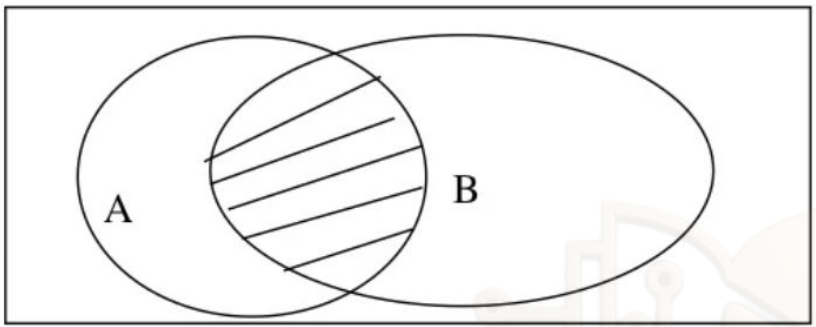
notes.arpanadhikari7.com.np

then, the probability of A given B is true is

$P(A|B) = 1/4$.

Available on notes.arpanadhikari7.com.np

How to Compute P(A|B)?



$$P(A \mid B) = \frac{N(A \text{ and } B)}{N(B)} = \frac{\frac{N(A \text{ and } B)}{N}}{\frac{N(B)}{N}} = \frac{P(A, B)}{P(B)}$$

notes.arpanadhikari7.com.np

$$P(\text{brown} \mid \text{cow}) = \frac{N(\text{brown-cows})}{N(\text{cows})} = \frac{P(\text{brown-cow})}{P(\text{cow})}$$

Available on notes.arpanadhikari7.com.np

Inference by enumeration

- Start with the joint probability distribution:

	toothache		¬ *toothache*	
	catch	¬ *catch*	*catch*	¬ *catch*
cavity	.108	.012	.072	.008
¬ *cavity*	.016	.064	.144	.576

- For any proposition φ, sum the atomic events where it is true: $P(\varphi) = \Sigma_{\omega:\omega \models \varphi} P(\omega)$

notes.arpanadhikari7.com.np

The full joint probability distribution specifies the probability of each complete assignment of values to random variables. It is usually too large to create or use in its explicit form, but when it is available it can be used to answer queries simply by adding up entries for the possible worlds corresponding to the query propositions.

Available on notes.arpanadhikari7.com.np

Inference by enumeration

- Start with the joint probability distribution:

	toothache		¬ *toothache*	
	catch	¬ *catch*	*catch*	¬ *catch*
cavity	.108	.012	.072	.008
¬ *cavity*	.016	.064	.144	.576

- For any proposition φ, sum the atomic events where it is true: $P(\varphi) = \Sigma_{\omega:\omega \models \varphi} P(\omega)$
- $P(toothache) = 0.108 + 0.012 + 0.016 + 0.064 = 0.2$

Available on notes.arpanadhikari7.com.np

Inference by enumeration

- Start with the joint probability distribution:

	toothache		¬ *toothache*	
	catch	¬ *catch*	*catch*	¬ *catch*
cavity	.108	.012	.072	.008
¬ *cavity*	.016	.064	.144	.576

- Can also compute conditional probabilities:

$$P(\neg cavity \mid toothache) = \frac{P(\neg cavity \wedge toothache)}{P(toothache)}$$

$$= \frac{0.016+0.064}{0.108 + 0.012 + 0.016 + 0.064}$$

$$= 0.4$$

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Example: Inference from the joint

	alarm		¬alarm	
	earthquake	¬earthquake	earthquake	¬earthquake
burglary	.01	.08	.001	.009
¬burglary	.01	.09	.01	.79

P(Burglary | alarm) = α P(Burglary, alarm)
= α [P(Burglary, alarm, earthquake) + P(Burglary, alarm, ¬earthquake)
= α [(.01, .01) + (.08, .09)]
= α [(.09, .1)]

Since P(burglary | alarm) + P(¬burglary | alarm) = 1, α = 1/(.09+.1) = 5.26
(i.e., P(alarm) = 1/α = .19 – **quizlet**: how can you verify this?)

P(burglary | alarm) = .09 * 5.26 = .474

P(¬burglary | alarm) = .1 * 5.26 = .526

Available on notes.arpanadhikari7.com.np

Independence: Definition

Events A and B are independent iff:

$$P(A, B) = P(A) \times P(B)$$

which is equivalent to

$$P(A|B) = P(A) \text{ and}$$

$$P(B|A) = P(B)$$

when $P(A, B) > 0$.

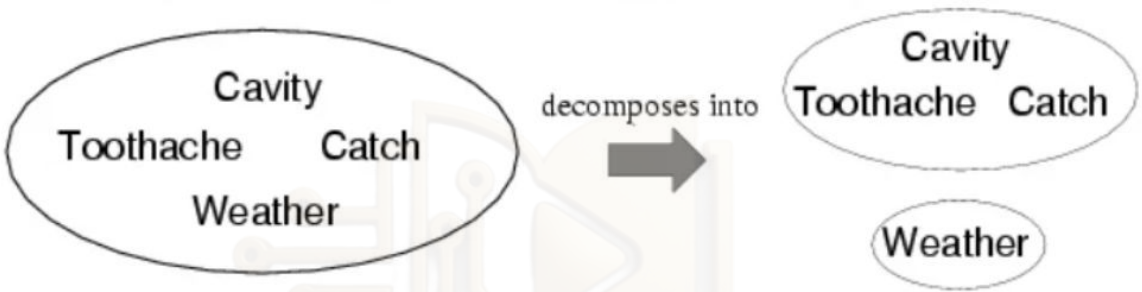
panadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Independence

- A and B are independent iff

$$\mathbf{P}(A|B) = \mathbf{P}(A) \quad \text{or } \mathbf{P}(B|A) = \mathbf{P}(B) \quad \text{or } \mathbf{P}(\text{A, B}) = \mathbf{P}(A)\,\mathbf{P}(B)$$



$$\mathbf{P}(\textit{Toothache, Catch, Cavity, Weather})$$
$$= \mathbf{P}(\textit{Toothache, Catch, Cavity})\,\mathbf{P}(\textit{Weather})$$

notes.arpanadhikari7.com.np

- Absolute independence is powerful but rare
- Dentistry is a large field with hundreds of variables, none of which are independent

Available on notes.arpanadhikari7.com.np

Conditional Independence

- Dependent events can become independent given certain other events.
- Example: Size of shoe, Age, Size of vocabulary
- Two events A, B are conditionally independent given a third event C iff

  P(A|B, C) = P(A|C) that is the probability of A is not changed after knowing C, given B is true.
- Equivalent formulations:

  P(E1, E2|E)=P(E1|E) P(E2|E)

  P(E2|E, E1)=P(E2|E)

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Conditional independence

- $\mathbf{P}(Toothache, Cavity, Catch)$ has $2^3 - 1 = 7$ independent entries
- If I have a cavity, the probability that the probe catches in it doesn't depend on whether I have a toothache:

  (1) $\mathbf{P}(catch \mid toothache, cavity) = \mathbf{P}(catch \mid cavity)$
- The same independence holds if I haven't got a cavity:

  (2) $\mathbf{P}(catch \mid toothache, \neg cavity) = \mathbf{P}(catch \mid \neg cavity)$
- *Catch* is conditionally independent of *Toothache* given *Cavity*:

  $\mathbf{P}(Catch \mid Toothache, Cavity) = \mathbf{P}(Catch \mid Cavity)$
- Equivalent statements:

  $\mathbf{P}(Toothache \mid Catch, Cavity) = \mathbf{P}(Toothache \mid Cavity)$

  $\mathbf{P}(Toothache, Catch \mid Cavity) = \mathbf{P}(Toothache \mid Cavity)\,\mathbf{P}(Catch \mid Cavity)$

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Conditional independence contd.

- Write out full joint distribution using chain rule:

$$\begin{aligned}
&\mathbf{P}(Toothache, Catch, Cavity) \\
&= \mathbf{P}(Toothache \mid Catch, Cavity)\, \mathbf{P}(Catch, Cavity) \\
&= \mathbf{P}(Toothache \mid Catch, Cavity)\, \mathbf{P}(Catch \mid Cavity)\, \mathbf{P}(Cavity) \\
&= \mathbf{P}(Toothache \mid Cavity)\, \mathbf{P}(Catch \mid Cavity)\, \mathbf{P}(\text{Cavity})
\end{aligned}$$

  i.e., 2 + 2 + 1 = 5 independent numbers

- In most cases, the use of conditional independence reduces the size of the representation of the joint distribution from exponential in n to linear in n.

- Conditional independence is our most basic and robust form of knowledge about uncertain environments.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Bayes' Rule

- Product rule $P(a \wedge b) = P(a \mid b)\, P(b) = P(b \mid a)\, P(a)$
  $\Rightarrow$ Bayes' rule: $P(a \mid b) = P(b \mid a)\, P(a) / P(b)$

- or in distribution form
  $$\mathbf{P}(Y|X) = \mathbf{P}(X|Y)\, \mathbf{P}(Y) / \mathbf{P}(X) = \alpha\, \mathbf{P}(X|Y)\, \mathbf{P}(Y)$$

- Useful for assessing diagnostic probability from causal probability:
  - $P(\text{Cause}|\text{Effect}) = P(\text{Effect}|\text{Cause})\, P(\text{Cause}) / P(\text{Effect})$

notes.arpanadhikari7.com.np

  - E.g., let *M* be meningitis, *S* be stiff neck:
    $P(m|s) = P(s|m)\, P(m) / P(s) = 0.8 \times 0.0001 / 0.1 = 0.0008$
  - Note: posterior probability of meningitis still very small!

Available on notes.arpanadhikari7.com.np

Bayes' Rule and conditional independence

$$\mathbf{P}(Cavity \mid toothache \wedge catch)$$
$$= \alpha\mathbf{P}(toothache \wedge catch \mid Cavity)\ \mathbf{P}(Cavity)$$
$$= \alpha\mathbf{P}(toothache \mid Cavity)\ \mathbf{P}(catch \mid Cavity)\ \mathbf{P}(Cavity)$$

- This is an example of a naïve Bayes model:

$$\mathbf{P}(\text{Cause},\text{Effect}_1, \ldots, \text{Effect}_n) = \mathbf{P}(\text{Cause})\ \pi_i \mathbf{P}(\text{Effect}_i \mid \text{Cause})$$



- Total number of parameters is linear in n

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Problems with Baye's rule

Baye's theorem is intractable

- Too many problems to be provided to acquire knowledge
- Space required to store all problems is too large
- Time required for processing is too much

Solution

- Attach certainty factors to rules
- Bayesian N/Ws

Available on notes.arpanadhikari7.com.np

The Baye's rule for multivalued variables: (representing a set of equations, each dealing with specific values of the variables)

$$\mathbf{P}(Y \mid X) = \frac{\mathbf{P}(X \mid Y)\mathbf{P}(Y)}{\mathbf{P}(X)}$$

notes.arpanadhikari7.com.np

Applying Bayes rule

We perceive as evidence the effect of some unknown cause, and we would like to determine that cause. Apply Baye's rule:

$$P(cause \mid effect) = \frac{P(effect \mid cause)P(cause)}{P(effect)} .$$

P(effect|cause) (e.g. P(symptoms|disease)) quantifies the relationship in the **causal direction**;

P(cause|effect) (e.g. P(disease|symptoms)) describes the **diagnostic direction**.

Available on notes.arpanadhikari7.com.np

Example

$$
\begin{aligned}
P(s \mid m) &= 0.7 \\
P(m) &= 1/50000 \\
P(s) &= 0.01 \\
P(m \mid s) &= \frac{P(s \mid m) P(m)}{P(s)} = \frac{0.7 \times 1/50000}{0.01} = 0.0014 \ .
\end{aligned}
$$

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

We can avoid assessing the prior probability of the evidence (P(s)) by instead computing a posterior probability for each value of the query variable (m and ¬m) and then normalizing the result.

$$\mathbf{P}(M \mid s) = \alpha \langle P(s \mid m)P(m), P(s \mid \neg m)P(\neg m) \rangle .$$

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

The general form of Bayes' rule with normalization:

$\mathbf{P}(Y|X) = \alpha \mathbf{P}(X|Y)\mathbf{P}(Y)$, where α is the normalization constant needed to make the entries in $\mathbf{P}(Y|X)$ sum to 1.

Bayes' rule allows unknown probabilities to be computed from known conditional probabilities, usually in the causal direction. Applying Bayes' rule with many pieces of evidence runs into the same scaling problems as does the full joint distribution.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

2. Using Bayes' rule: combining evidence

The general definition of **conditional independence** of two variable X and Y, given a third variable Z is:

$\mathbf{P}(X, Y|Z) = \mathbf{P}(X|Z)\mathbf{P}(Y|Z)$

The equivalent forms can also be used:

$\mathbf{P}(X|Y, Z) = \mathbf{P}(X|Z)$ and $\mathbf{P}(Y|X, Z) = P(Y|Z)$

e.g.

assert conditional independence of the variables Toothache and Catch, given Cavity:

$\mathbf{P}(\text{Toothache}, \text{Catch}|\text{Cavity}) = \mathbf{P}(\text{Toothache}|\text{Cavity})\mathbf{P}(\text{Catch}|\text{Cavity})$.

Absolute independence assertions allow a decomposition of the full joint distribution into much smaller pieces, the same is true for conditional independence assertions.

Available on notes.arpanadhikari7.com.np

Naïve Bayes model: In which a single cause directly influences a number of effects, all of which are conditionally independent, given the cause. The full joint distribution can be written as

$$\mathbf{P}(Cause, Effect_1, \ldots, Effect_n) = \mathbf{P}(Cause) \prod_i \mathbf{P}(Effect_i \mid Cause) .$$

Conditional independence brought about by direct causal relationships in the domain might allow the full joint distribution to be factored into smaller, conditional distributions. The **naïve Bayes** model assumes the conditional independence of all effect variables, given a single cause variable, and grow linearly with the number of effects.

Available on notes.arpanadhikari7.com.np

Uses

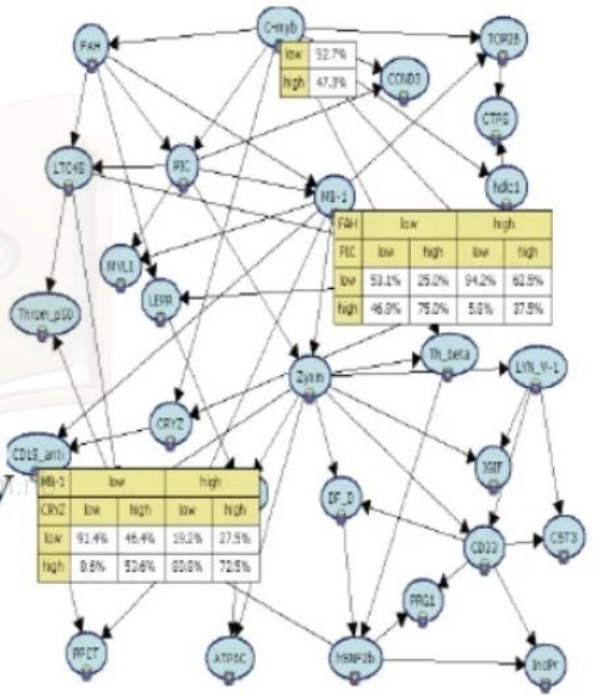
Bayes' Rule has use cases in many areas:

- Understanding probability problems (including those in medical research)
- Statistical modelling and inference
- Machine learning algorithms (such as Naive Bayes, Expectation Maximisation)
- Quantitative modelling and forecasting

notes.arpanadhikari7.com.np

Bayesian Networks

- A compact representation of a joint probability of variables on the basis of the concept of conditional independence.
- Qualitative part: graph theory
  - Directed acyclic graph **(DAG)**
  - Nodes: variables
  - Edges: dependency or influence of a variable on another.
- Quantitative part: probability theory
  - Set of conditional probabilities for all variables
- Naturally handles the problem of complexity and uncertainty.



Available on notes.arpanadhikari7.com.np

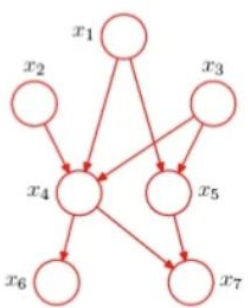
Bayesian Networks

- The joint distribution defined by a graph is given by the product of a conditional distribution of each node conditioned on their parent nodes.

$$p(\mathbf{x}) = \prod_{k=1}^{K} p(x_k \mid Pa(x_k))$$

($Pa(x_k)$ denotes the set of parents of x_k)

ex)



$$p(x_1, x_2, \ldots, x_7) =$$

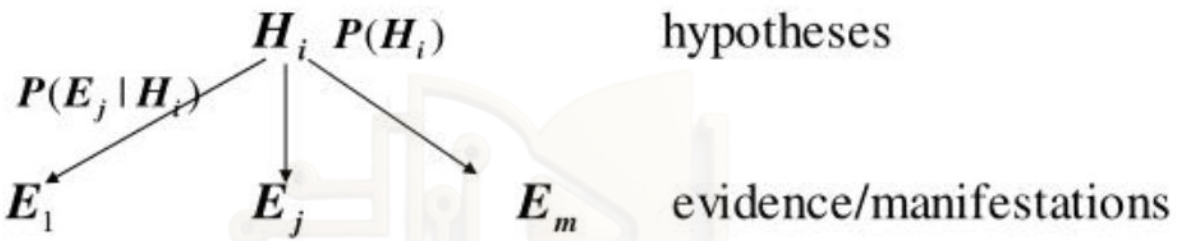
$$p(x_1)p(x_2)p(x_3)p(x_4|x_1, x_2, x_3)p(x_5|x_1, x_3)p(x_6|x_4)p(x_7|x_4, x_5).$$

* Without given DAG structure, usual chain rule can be applied to get the joint distribution. But computational cost is much higher.

Available on notes.arpanadhikari7.com.np

Bayesian Inference

- In the setting of diagnostic/evidential reasoning



  - Know prior probability of hypothesis $P(H_i)$
    conditional probability $P(E_j \mid H_i)$
  - Want to compute the *posterior probability* $P(H_i \mid E_j)$

- Bayes's theorem (formula 1):

$$P(H_i \mid E_j) = P(H_i)P(E_j \mid H_i) / P(E_j)$$

Available on notes.arpanadhikari7.com.np

Simple Bayesian diagnostic reasoning

- Knowledge base:
  - Evidence / manifestations: $E_1, \ldots E_m$
  - Hypotheses / disorders: $H_1, \ldots H_n$
    - E_j and H_i are **binary**; hypotheses are **mutually exclusive** (non-overlapping) and **exhaustive** (cover all possible cases)
  - Conditional probabilities: $P(E_j \mid H_i)$, $i = 1, \ldots n$; $j = 1, \ldots m$
- Cases (evidence for a particular instance): $E_1, \ldots, E_l$
- Goal: Find the hypothesis H_i with the highest posterior
  - $\text{Max}_i\ P(H_i \mid E_1, \ldots, E_l)$

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Bayesian diagnostic reasoning II

- Bayes' rule says that
  - $P(H_i \mid E_1, \ldots, E_l) = P(E_1, \ldots, E_l \mid H_i)\, P(H_i) / P(E_1, \ldots, E_l)$
- Assume each piece of evidence E_i is conditionally independent of the others, *given* a hypothesis H_i, then:
  - $P(E_1, \ldots, E_l \mid H_i) = \prod_{j=1}^{l} P(E_j \mid H_i)$
- If we only care about relative probabilities for the H_i, then we have:
  - $P(H_i \mid E_1, \ldots, E_l) = \alpha\, P(H_i) \prod_{j=1}^{l} P(E_j \mid H_i)$

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Simple Bayesian Network

$S \in \{no, light, heavy\}$



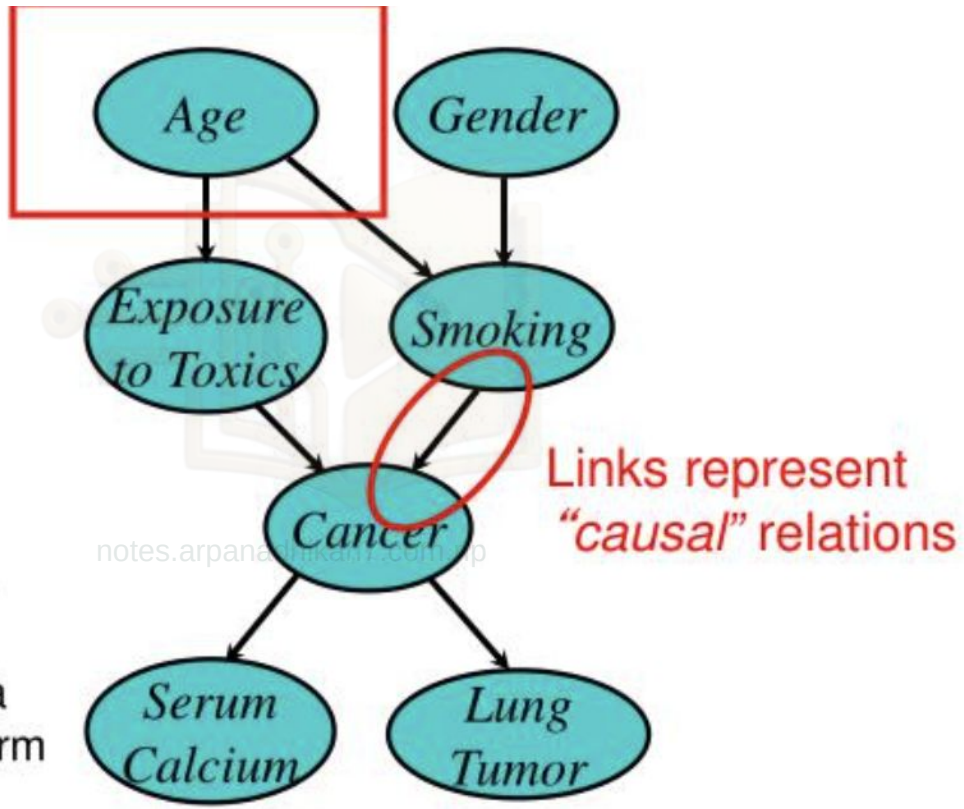
$P(S=no)$	0.80
$P(S=light)$	0.15
$P(S=heavy)$	0.05

$C \in \{none, benign, malignant\}$

$Smoking=$	no	$light$	$heavy$
$P(C=none)$	0.96	0.88	0.60
$P(C=benign)$	0.03	0.08	0.25
$P(C=malig)$	0.01	0.04	0.15

Available on notes.arpanadhikari7.com.np

Complex Bayesian Network



- Does gender cause smoking?
- Influence might be a more appropriate term

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np